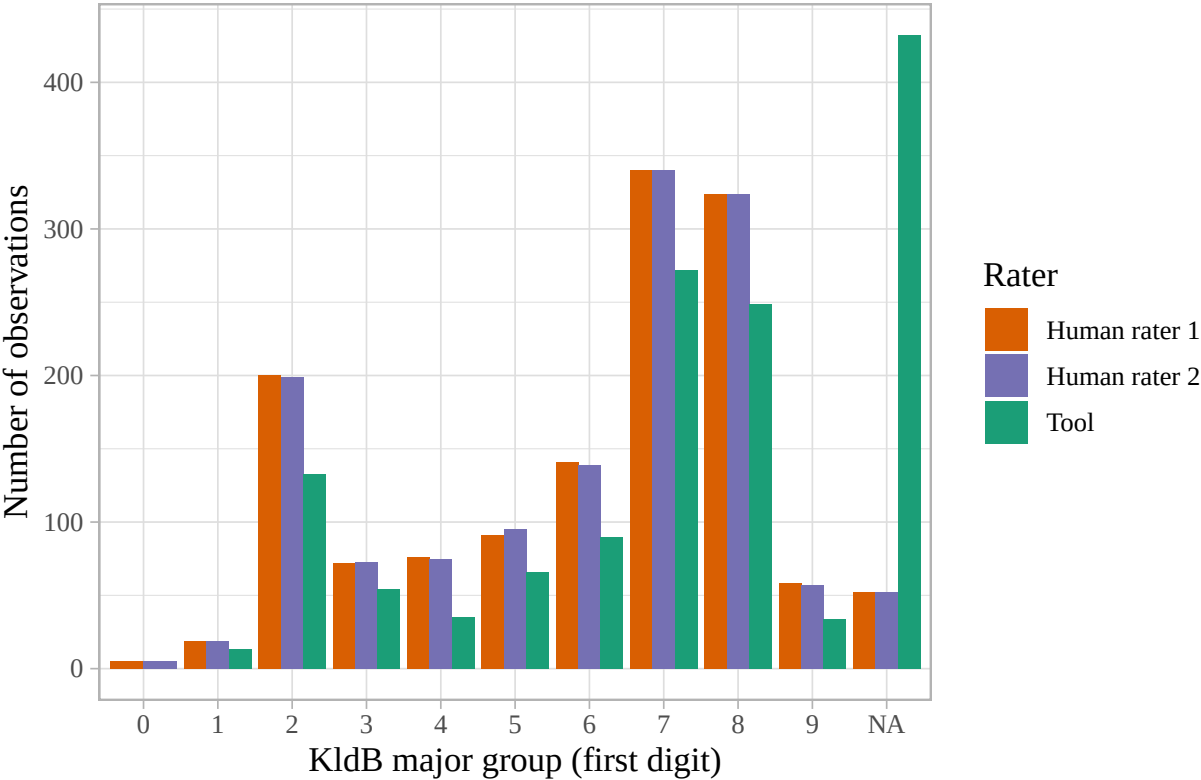
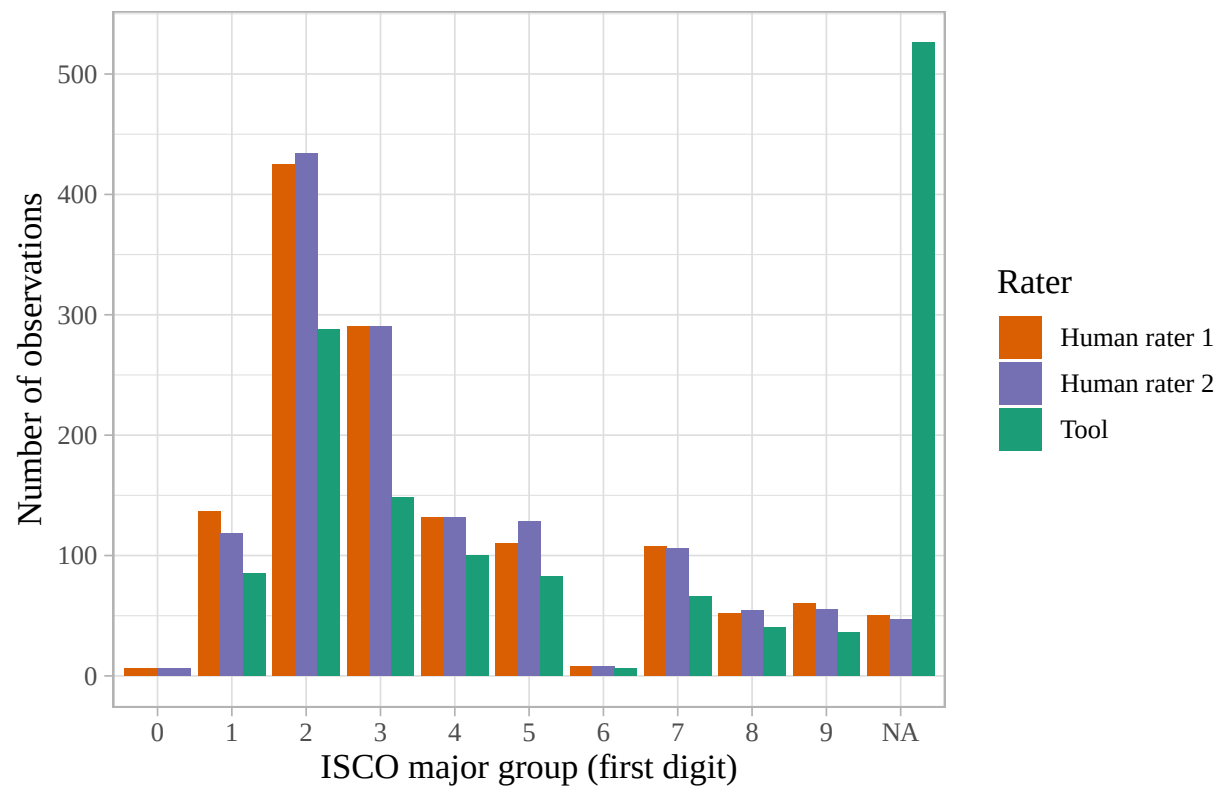


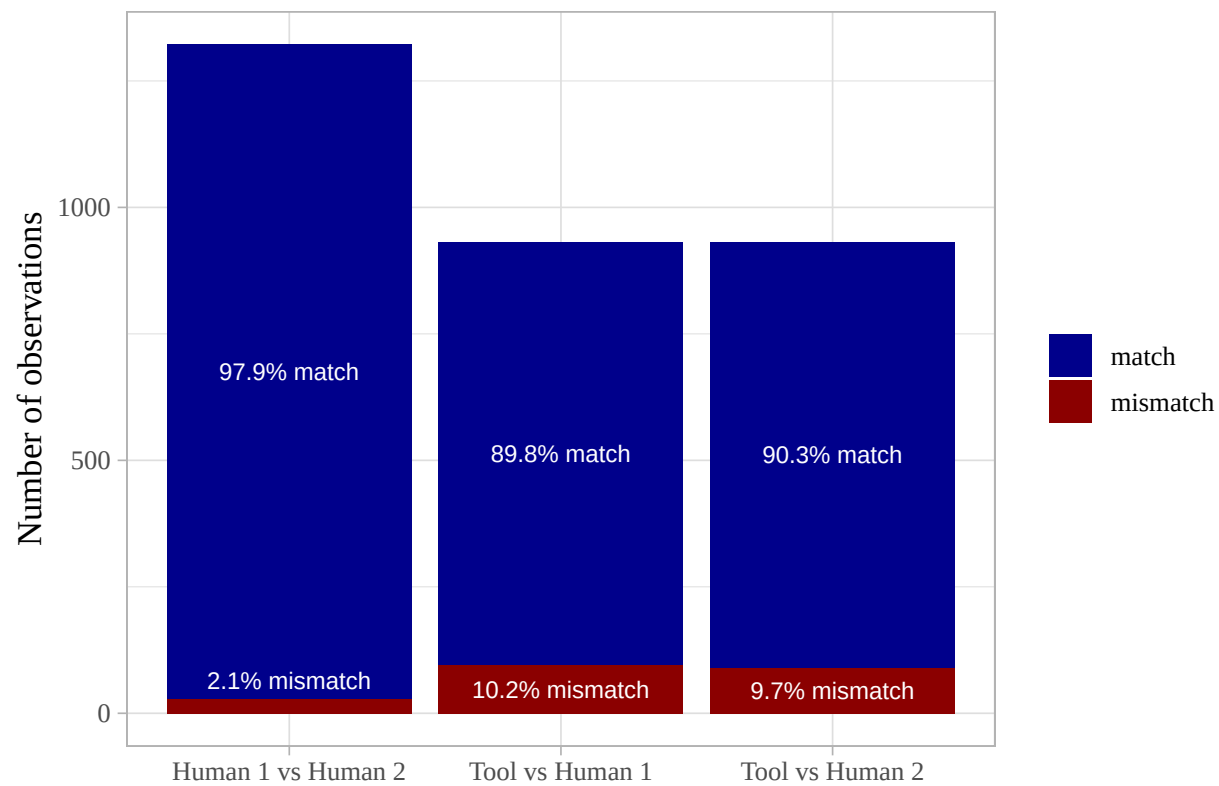
ISCO Alpha Beta Parameter Comparison

```
## |  
## [1] "C:/Users/jeroe/Uni_C/noisy_measurement/src/data_preparation.R"
```

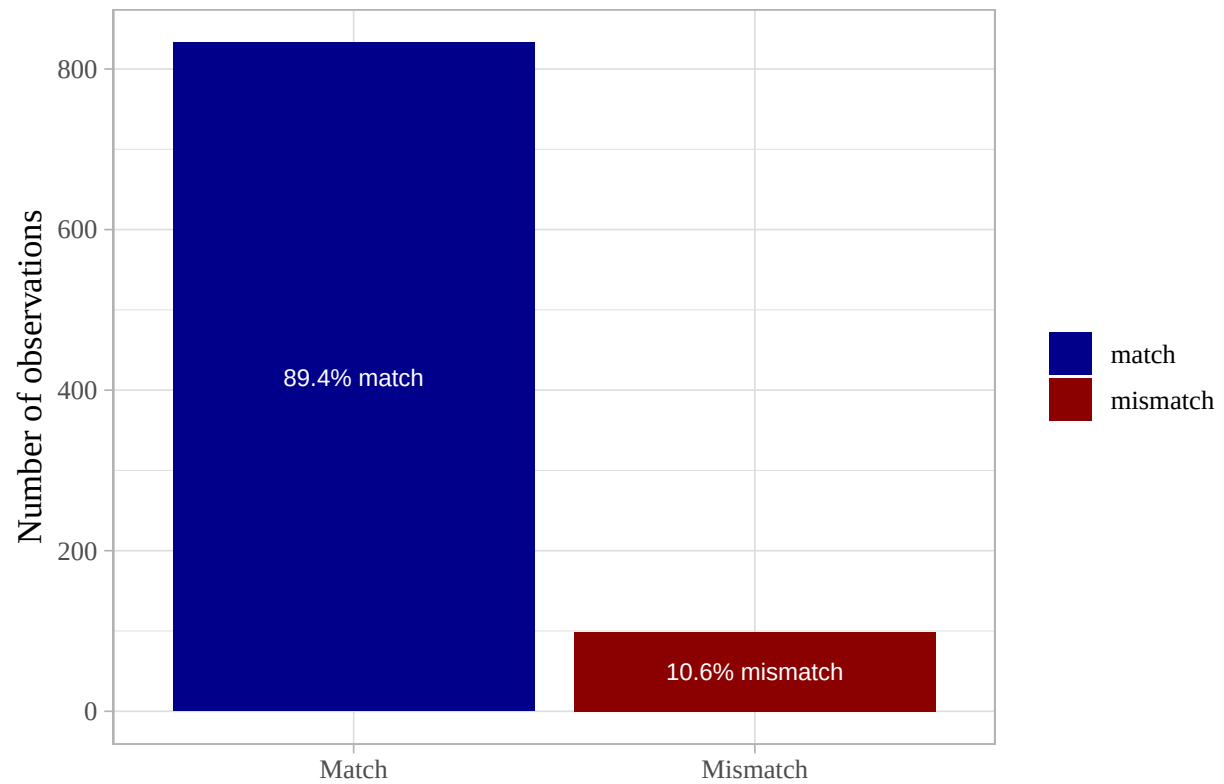




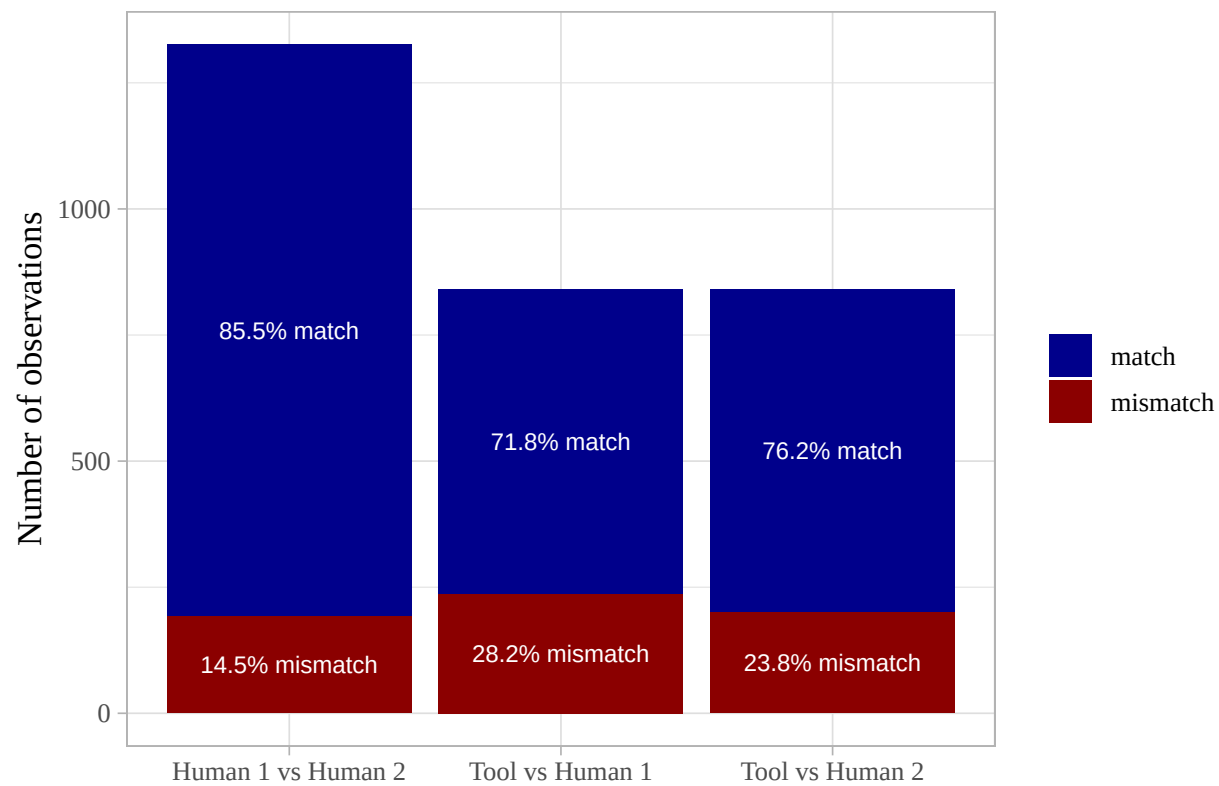
KldB: Pairwise agreement on the first digit



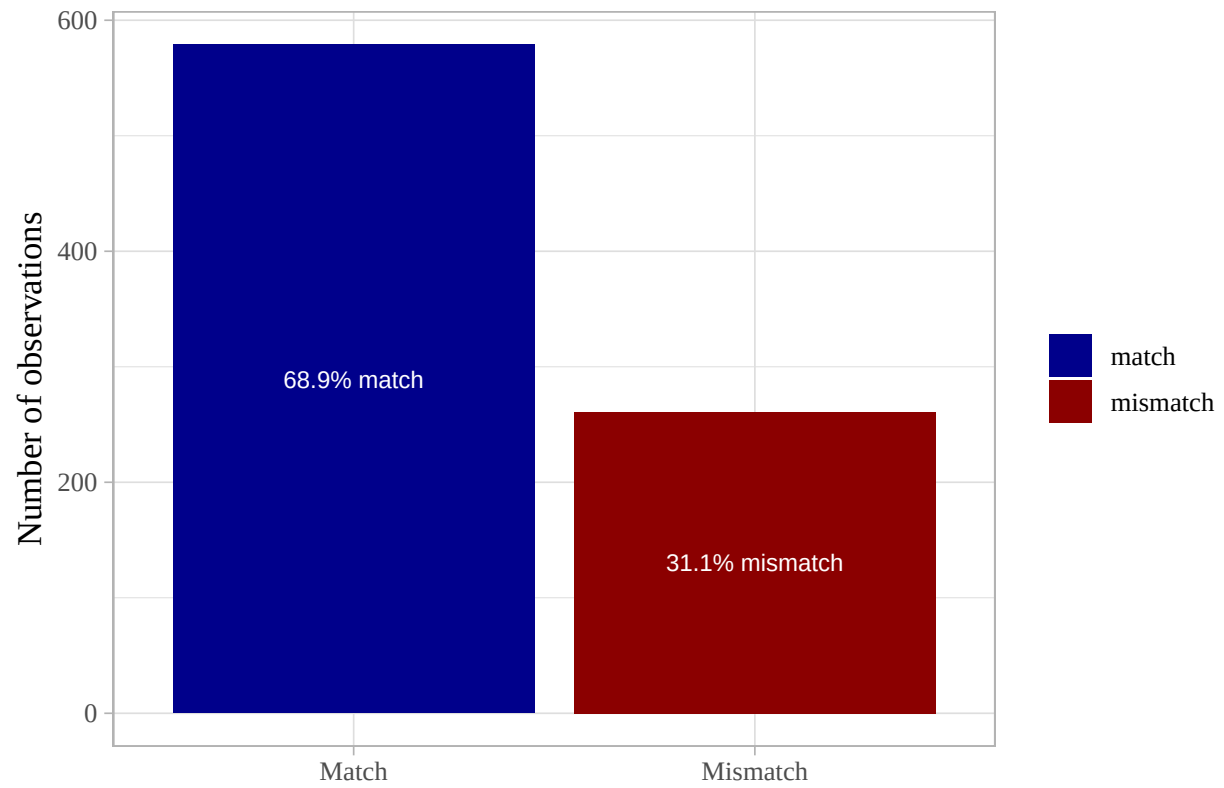
KldB: Agreement across all three sources on the first digit

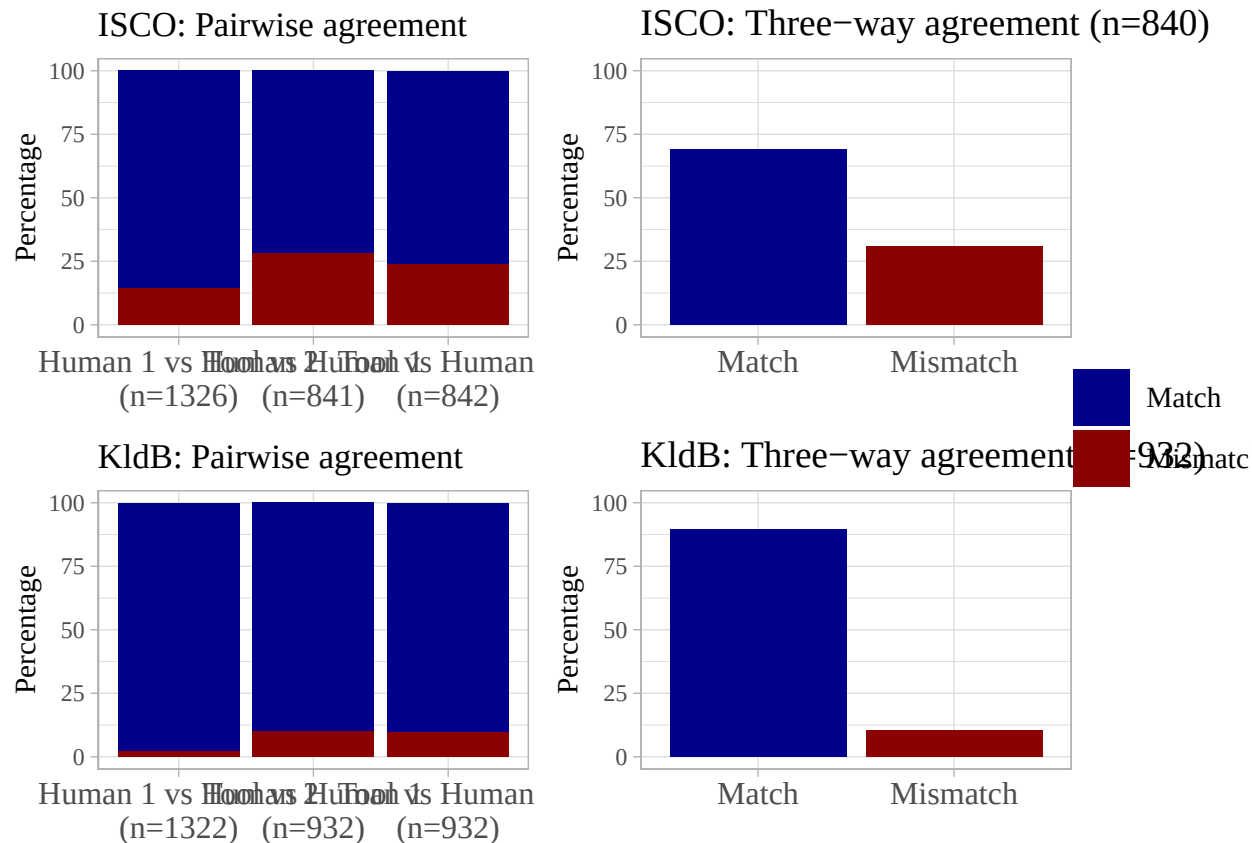


ISCO: Pairwise agreement on the first digit



ISCO: Agreement across all three sources on the first digit





```
# Packages
library(data.table)
library(dplyr)
library(tidyrr)
library(cmdstanr)

# keep only relevant columns
isco_data <- data_basis %>%
  select(item_id, isco_tool_1st, res1_isco_1st, res2_isco_1st)

#Rename columns
setnames(
  data_basis,
  old = c("isco_tool_1st", "res1_isco_1st", "res2_isco_1st"),
  new = c("isco_tool", "isco_human1", "isco_human2")
)

head(data_basis)
```

```
##      kldb_tool_vorlaufig kldb_tool_mit_anfniveau res1_kldb res2_kldb isco_tool_vorlaufig isco_tool_mit.
##      <char> <char> <char> <char> <char> <char>
## 1:      81883      81883      81883      81883      2433
## 2:      52112      52112      52112      52112      8322
## 3:      <NA>      <NA>      83131      83131      <NA>
```

```

## 4:          23411          23411    23411    23411          9329
## 5:          72134          72134    72132    72132          2412
## 6:          25104          25104    27224    27224          2144
##   res1_isco res2_isco kldb_infas_num          kldb_infas_str isco_infas_n
##   <char>    <char>    <char>          <char>          <char>
## 1:      2433      2433      81883      [81883] Pharmazie (s.s.T.) - Spezialist      24
## 2:      8322      8322      52112      [52112] Berufskraftfahrer(Pers./PKW.)-Fachkraft      83
## 3:      3412      3412      83131      [83131] Heilerziehungspflege, Sonderpaed. -Helfer      34
## 4:      7322      7322      23412          [23412] Drucktechnik - Fachkraft      73
## 5:      4312      4312      72132          [72132] Versicherungskaufleute - Fachkraft      43
## 6:      2149      2144      27224      [27224] Konstruktion und Geraetebau - Experte      21
##
##
## 1: [2433] Akademische und vergleichbare Fachkräfte im Bereich Vertrieb (Technik und Medizin, ohne In
## 2:                                     [8322] Personenkraftwagen-, T
## 3:                                     [3412] Nicht
## 4:
## 5:                                     [4312] Bürokräfte in der St
## 6:
##   kldb_tool_1st res1_kldb_1st res2_kldb_1st isco_tool isco_human1 isco_human2 item_id
##   <char>        <char>        <char>    <char>    <char>    <char>    <int>
## 1:           8           8           8         2         2         2         1
## 2:           5           5           5         8         8         8         2
## 3:        <NA>           8           8        <NA>         3         3         3
## 4:           2           2           2         9         7         7         4
## 5:           7           7           7         2         4         4         5
## 6:           2           2           2         2         2         2         6

```

```
#### 1) Select only ISCO, as character
```

```
isco_data <- data_basis %>%
  dplyr::select(item_id, isco_human1, isco_human2, isco_tool) %>%
  mutate(across(c(isco_human1, isco_human2, isco_tool), as.character))
```

```
### Remove all lines with NAs
```

```
isco_data <- isco_data %>%
  filter(!is.na(isco_human1) & !is.na(isco_human2) & !is.na(isco_tool))
```

```
# Keep only three categories: "2" and "3" and "1"
```

```
isco_data <- isco_data %>%
  filter(isco_human1 == "2" | isco_human1 == "3" | isco_human1 == "1") %>%
  filter(isco_human2 == "2" | isco_human2 == "3" | isco_human2 == "1") %>%
  filter(isco_tool == "2" | isco_tool == "3" | isco_tool == "1")
```

```
# 1) Number of rows where all three classify as "2"
```

```
count_all_2 <- isco_data %>%
  dplyr::filter(isco_human1 == "2", isco_human2 == "2", isco_tool == "2") %>%
  nrow()
```

```
# 2) Number of rows where all three classify as "3"
```

```
count_all_3 <- isco_data %>%
  dplyr::filter(isco_human1 == "3", isco_human2 == "3", isco_tool == "3") %>%
  nrow()
```

```
# 2a) Number of rows where all three classify as "1"
```



```

count_all_1 <- isco_data %>%
  dplyr::filter(isco_human1 == "1", isco_human2 == "1", isco_tool == "1") %>%
  nrow()

# 3) Number of rows where at least one value is "2" and at least one value is "3"
count_mixed_2_and_3 <- isco_data %>%
  dplyr::rowwise() %>%
  dplyr::filter(
    any(c(isco_human1, isco_human2, isco_tool) == "2") &
    any(c(isco_human1, isco_human2, isco_tool) == "3")
  ) %>%
  nrow()

# 3a) Number of rows where at least one value is "1" and at least one value is "2" or "3"
count_mixed_1_and_else <- isco_data %>%
  dplyr::rowwise() %>%
  dplyr::filter(
    any(c(isco_human1, isco_human2, isco_tool) == "1") &
    any(c(isco_human1, isco_human2, isco_tool) != "1")
  ) %>%
  nrow()

# 4) Number of rows where minimum one element is neither "2" nor "3" nor "1" - should be 0
count_other <- isco_data %>%
  dplyr::rowwise() %>%
  dplyr::filter(
    any(!c(isco_human1, isco_human2, isco_tool) %in% c("1", "2", "3"))
  ) %>%
  nrow()

# Show results
count_all_1          # 46

## [1] 46
count_all_2          # 235

## [1] 235
count_all_3          # 80

## [1] 80
count_mixed_2_and_3 # 42

## [1] 49
count_mixed_1_and_else #57

## [1] 57
count_other          # 0

## [1] 0

# Reindex item_id (1, 2, 3,...)
isco_data <- isco_data %>%
  dplyr::mutate(item_id = dplyr::row_number())

```

```

# Collect all observed classes
isco_levels <- isco_data %>%
  dplyr::select(isco_human1, isco_human2, isco_tool) %>%
  unlist(use.names = FALSE) %>%
  unique() %>%
  sort() %>%
  as.character() %>%
  as.vector()

# K, number of distinct classes
K <- length(isco_levels)
stopifnot(K >= 3)

# Helper: map original labels to 1..K using isco_levels for stan compatibility
to_index <- function(v) as.integer(factor(v, levels = isco_levels))

# 1) Map original labels to 1..K
isco_data_idx <- isco_data %>%
  dplyr::mutate(
    isco_human1 = to_index(isco_human1),
    isco_human2 = to_index(isco_human2),
    isco_tool   = to_index(isco_tool)
  )

# 2) Long format (now without NAs)
long_data <- isco_data_idx %>%
  dplyr::transmute(item_id, r1 = isco_human1, r2 = isco_human2, r3 = isco_tool) %>%
  tidyr::pivot_longer(cols = starts_with("r"), names_to = "rater", values_to = "y") %>%
  dplyr::mutate(rater_id = as.integer(factor(rater, levels = c("r1", "r2", "r3")))) %>%
  dplyr::arrange(item_id, rater_id)

# Robust 1..I indexing
item_ids <- sort(unique(long_data$item_id))
long_data$item_id_idx <- as.integer(factor(long_data$item_id, levels = item_ids))

I <- length(item_ids) # number of items (people interviewed)
J <- 3L # number of raters
N <- nrow(long_data) # number of observations (I * J if no missing)

cat("Items after filtering:", I, "\n")

## Items after filtering: 460
cat("Observations:", N, "\n")

## Observations: 1380
cat("Classes:", K, "\n")

## Classes: 3
cat("Class levels:", paste(isco_levels, collapse = ", "), "\n")

## Class levels: 1, 2, 3

```

```

# Packages
library(data.table)
library(dplyr)
library(tidyr)
library(cmdstanr)

# 3) Write Stan model to file
stan_file <- "dawid_skene_isco.stan"
writeLines(con = stan_file, text = '
  data {
    int<lower=2> K;           // number of categories
    int<lower=1> I;           // items
    int<lower=1> J;           // raters
    int<lower=1> N;           // number of observed ratings
    array[N] int<lower=1, upper=I> ii; // item index per observation
    array[N] int<lower=1, upper=J> jj; // rater index per observation
    array[N] int<lower=1, upper=K> y;  // observed category per observation

    vector<lower=0>[K] alpha;           // prior for pi
    array[K] vector<lower=0>[K] beta;   // Dirichlet priors for theta rows, vary by true class
  }

  parameters {
    simplex[K] pi;                     // prevalence over true classes
    array[J, K] simplex[K] theta;      // confusion rows: for rater j and true class k
  }

  transformed parameters {
    // log_q_z[i, k] = log p(z_i = k | pi, theta) up to a constant
    array[I] vector[K] log_q_z;

    // Initialize with log pi
    for (i in 1:I) {
      log_q_z[i] = log(pi);
    }

    // Accumulate per observation
    for (n in 1:N) {
      int i = ii[n];
      int j = jj[n];
      int y_n = y[n];
      for (k in 1:K) {
        // Add log categorical pmf of y_n under theta[j, k, :]
        log_q_z[i, k] += categorical_lpmf(y_n | theta[j, k]);
      }
    }
  }

  model {
    // Priors
    pi ~ dirichlet(alpha);

```

```

for (j in 1:J)
  for (k in 1:K)
    theta[j, k] ~ dirichlet(beta[k]);

// Marginalized likelihood
for (i in 1:I) {
  target += log_sum_exp(log_q_z[i]);
}
}

generated quantities {
  array[I] simplex[K] q_z;
  for (i in 1:I) {
    vector[K] lq = log_q_z[i];
    q_z[i] = softmax(lq);
  }
}
')

# Compile model once
mod <- cmdstan_model(stan_file, cpp_options = list(stan_threads = TRUE))

# Define 4 model configurations
model_configs <- list(
  list(
    name = "model1",
    alpha = c(1,1,1),
    beta = lapply(1:K, function(k) { v <- rep(1, K); v[k] <- 15; v })
  ),
  list(
    name = "model2",
    alpha = c(7,20,15),
    beta = lapply(1:K, function(k) { v <- rep(1, K); v[k] <- 15; v })
  )
# list(
#   name = "model3",
#   alpha = c(7,20,15),
#   beta = lapply(1:K, function(k) { v <- rep(1, K); v[k] <- 30; v })
# ),
# list(
#   name = "model4",
#   alpha = c(1,1,1),
#   beta = lapply(1:K, function(k) { v <- rep(1, K); v[k] <- 30; v })
# )
# list(
#   name = "alpha11_beta5diag",
#   alpha = c(1.0, 1.0),
#   beta = matrix(1.0, nrow = K, ncol = K) + diag(4.0, K) # 5 on diagonal, 1 elsewhere
# ),
# list(
#   name = "alpha1510_beta1",
#   alpha = c(15.0, 10.0),
#   beta = matrix(1.0, nrow = K, ncol = K)

```

```

# # ),
# list(
#   name = "alpha1510_beta5diag",
#   alpha = c(15.0, 10.0),
#   beta = matrix(1.0, nrow = K, ncol = K) + diag(4.0, K) # 5 on diagonal, 1 elsewhere
# )
)

# Fit all 1 models
fits <- list()
for (config in model_configs) {
  cat("\n=== Fitting model:", config$name, "===\n")

  stan_data <- list(
    I = I,
    J = J,
    K = K,
    N = N,
    ii = as.integer(long_data$item_id_idx),
    jj = as.integer(long_data$rater_id),
    y = as.integer(long_data$y),
    alpha = config$alpha,
    beta = config$beta
  )

  fit <- mod$sample(
    data = stan_data,
    seed = 1234,
    chains = 4,
    parallel_chains = 4,
    threads_per_chain = 1,
    iter_warmup = 1000,
    iter_sampling = 4000,
    refresh = 200,
    adapt_delta = 0.99
  )

  fits[[config$name]] <- fit

  # Check convergence
  s <- fit$summary()
  s$rhat <- as.numeric(s$rhat)
  bad_rhat <- subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))
  if (nrow(bad_rhat) > 0) {
    cat("WARNING: R-hat values above 1.1:\n")
    print(bad_rhat[, c("variable", "rhat", "ess_bulk", "ess_tail")])
  } else {
    cat("No R-hat values above 1.1\n")
  }
}

##
## === Fitting model: model1 ===
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

```

##
## Chain 1 Iteration:      1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:      1 / 5000 [ 0%] (Warmup)
## Chain 3 Iteration:      1 / 5000 [ 0%] (Warmup)
## Chain 4 Iteration:      1 / 5000 [ 0%] (Warmup)
## Chain 2 Iteration:    200 / 5000 [ 4%] (Warmup)
## Chain 3 Iteration:    200 / 5000 [ 4%] (Warmup)
## Chain 1 Iteration:    200 / 5000 [ 4%] (Warmup)
## Chain 4 Iteration:    200 / 5000 [ 4%] (Warmup)
## Chain 2 Iteration:    400 / 5000 [ 8%] (Warmup)
## Chain 3 Iteration:    400 / 5000 [ 8%] (Warmup)
## Chain 1 Iteration:    400 / 5000 [ 8%] (Warmup)
## Chain 4 Iteration:    400 / 5000 [ 8%] (Warmup)
## Chain 2 Iteration:    600 / 5000 [12%] (Warmup)
## Chain 3 Iteration:    600 / 5000 [12%] (Warmup)
## Chain 1 Iteration:    600 / 5000 [12%] (Warmup)
## Chain 4 Iteration:    600 / 5000 [12%] (Warmup)
## Chain 2 Iteration:    800 / 5000 [16%] (Warmup)
## Chain 3 Iteration:    800 / 5000 [16%] (Warmup)
## Chain 1 Iteration:    800 / 5000 [16%] (Warmup)
## Chain 4 Iteration:    800 / 5000 [16%] (Warmup)
## Chain 2 Iteration:   1000 / 5000 [20%] (Warmup)
## Chain 2 Iteration:   1001 / 5000 [20%] (Sampling)
## Chain 3 Iteration:   1000 / 5000 [20%] (Warmup)
## Chain 3 Iteration:   1001 / 5000 [20%] (Sampling)
## Chain 4 Iteration:   1000 / 5000 [20%] (Warmup)
## Chain 4 Iteration:   1001 / 5000 [20%] (Sampling)
## Chain 1 Iteration:   1000 / 5000 [20%] (Warmup)
## Chain 1 Iteration:   1001 / 5000 [20%] (Sampling)
## Chain 2 Iteration:   1200 / 5000 [24%] (Sampling)
## Chain 3 Iteration:   1200 / 5000 [24%] (Sampling)
## Chain 4 Iteration:   1200 / 5000 [24%] (Sampling)
## Chain 2 Iteration:   1400 / 5000 [28%] (Sampling)
## Chain 1 Iteration:   1200 / 5000 [24%] (Sampling)
## Chain 4 Iteration:   1400 / 5000 [28%] (Sampling)
## Chain 2 Iteration:   1600 / 5000 [32%] (Sampling)
## Chain 3 Iteration:   1400 / 5000 [28%] (Sampling)
## Chain 1 Iteration:   1400 / 5000 [28%] (Sampling)
## Chain 2 Iteration:   1800 / 5000 [36%] (Sampling)
## Chain 4 Iteration:   1600 / 5000 [32%] (Sampling)
## Chain 3 Iteration:   1600 / 5000 [32%] (Sampling)
## Chain 1 Iteration:   1600 / 5000 [32%] (Sampling)
## Chain 2 Iteration:   2000 / 5000 [40%] (Sampling)
## Chain 4 Iteration:   1800 / 5000 [36%] (Sampling)
## Chain 1 Iteration:   1800 / 5000 [36%] (Sampling)
## Chain 3 Iteration:   1800 / 5000 [36%] (Sampling)
## Chain 2 Iteration:   2200 / 5000 [44%] (Sampling)
## Chain 4 Iteration:   2000 / 5000 [40%] (Sampling)
## Chain 1 Iteration:   2000 / 5000 [40%] (Sampling)
## Chain 3 Iteration:   2000 / 5000 [40%] (Sampling)
## Chain 2 Iteration:   2400 / 5000 [48%] (Sampling)
## Chain 4 Iteration:   2200 / 5000 [44%] (Sampling)
## Chain 1 Iteration:   2200 / 5000 [44%] (Sampling)
## Chain 3 Iteration:   2200 / 5000 [44%] (Sampling)

```

```

## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 120.6 seconds.
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 131.8 seconds.
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)

```

```

## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 139.4 seconds.
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 143.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 133.9 seconds.
## Total execution time: 143.9 seconds.
##
## No R-hat values above 1.1
##
## === Fitting model: model2 ===
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 2 Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 3 Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 4 Iteration:    1 / 5000 [  0%] (Warmup)
## Chain 1 Iteration:   200 / 5000 [  4%] (Warmup)
## Chain 2 Iteration:   200 / 5000 [  4%] (Warmup)
## Chain 3 Iteration:   200 / 5000 [  4%] (Warmup)
## Chain 4 Iteration:   200 / 5000 [  4%] (Warmup)
## Chain 1 Iteration:   400 / 5000 [  8%] (Warmup)
## Chain 2 Iteration:   400 / 5000 [  8%] (Warmup)
## Chain 3 Iteration:   400 / 5000 [  8%] (Warmup)
## Chain 1 Iteration:   600 / 5000 [ 12%] (Warmup)
## Chain 2 Iteration:   600 / 5000 [ 12%] (Warmup)
## Chain 4 Iteration:   400 / 5000 [  8%] (Warmup)
## Chain 3 Iteration:   600 / 5000 [ 12%] (Warmup)
## Chain 2 Iteration:   800 / 5000 [ 16%] (Warmup)
## Chain 1 Iteration:   800 / 5000 [ 16%] (Warmup)
## Chain 4 Iteration:   600 / 5000 [ 12%] (Warmup)
## Chain 3 Iteration:   800 / 5000 [ 16%] (Warmup)
## Chain 2 Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 2 Iteration:  1001 / 5000 [ 20%] (Sampling)
## Chain 1 Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 1 Iteration:  1001 / 5000 [ 20%] (Sampling)
## Chain 4 Iteration:   800 / 5000 [ 16%] (Warmup)
## Chain 3 Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 3 Iteration:  1001 / 5000 [ 20%] (Sampling)
## Chain 2 Iteration:  1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration:  1200 / 5000 [ 24%] (Sampling)
## Chain 4 Iteration:  1000 / 5000 [ 20%] (Warmup)
## Chain 4 Iteration:  1001 / 5000 [ 20%] (Sampling)
## Chain 3 Iteration:  1200 / 5000 [ 24%] (Sampling)
## Chain 2 Iteration:  1400 / 5000 [ 28%] (Sampling)
## Chain 1 Iteration:  1400 / 5000 [ 28%] (Sampling)
## Chain 3 Iteration:  1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration:  1600 / 5000 [ 32%] (Sampling)
## Chain 4 Iteration:  1200 / 5000 [ 24%] (Sampling)
## Chain 1 Iteration:  1600 / 5000 [ 32%] (Sampling)
## Chain 3 Iteration:  1600 / 5000 [ 32%] (Sampling)
## Chain 2 Iteration:  1800 / 5000 [ 36%] (Sampling)

```



```

## Chain 1 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 4 Iteration: 1400 / 5000 [ 28%] (Sampling)
## Chain 2 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 3 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 1 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 3 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 4 Iteration: 1600 / 5000 [ 32%] (Sampling)
## Chain 1 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 3 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 4 Iteration: 1800 / 5000 [ 36%] (Sampling)
## Chain 2 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 5000 [ 40%] (Sampling)
## Chain 2 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 3 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 1 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 2 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 4 Iteration: 2200 / 5000 [ 44%] (Sampling)
## Chain 2 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 3 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 1 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 4 Iteration: 2400 / 5000 [ 48%] (Sampling)
## Chain 2 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 3 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 2 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 3 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 2600 / 5000 [ 52%] (Sampling)
## Chain 1 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 2 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 3 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 1 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 2 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 2800 / 5000 [ 56%] (Sampling)
## Chain 3 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 1 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 2 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 3 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 3000 / 5000 [ 60%] (Sampling)
## Chain 1 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 2 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 3 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 1 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 2 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3200 / 5000 [ 64%] (Sampling)
## Chain 3 Iteration: 4600 / 5000 [ 92%] (Sampling)

```

```

## Chain 1 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 2 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 2 finished in 129.8 seconds.
## Chain 3 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 3400 / 5000 [ 68%] (Sampling)
## Chain 1 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 3 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 3 finished in 137.3 seconds.
## Chain 1 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 1 finished in 141.0 seconds.
## Chain 4 Iteration: 3600 / 5000 [ 72%] (Sampling)
## Chain 4 Iteration: 3800 / 5000 [ 76%] (Sampling)
## Chain 4 Iteration: 4000 / 5000 [ 80%] (Sampling)
## Chain 4 Iteration: 4200 / 5000 [ 84%] (Sampling)
## Chain 4 Iteration: 4400 / 5000 [ 88%] (Sampling)
## Chain 4 Iteration: 4600 / 5000 [ 92%] (Sampling)
## Chain 4 Iteration: 4800 / 5000 [ 96%] (Sampling)
## Chain 4 Iteration: 5000 / 5000 [100%] (Sampling)
## Chain 4 finished in 198.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 151.6 seconds.
## Total execution time: 198.4 seconds.
##
## No R-hat values above 1.1
fit_basis <- fits[["model1"]]

if (interactive() && is.null(knitr::opts_knit$get("rmarkdown.pandoc.to"))) {
  library(shinystan)
  launch_shinystan(fit_basis)
}

library(dplyr)
library(posterior)
library(ggplot2)
library(reshape2)

out_dir <- "img/model_results/isco/three_categories/alpha1"
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

# use first fit
fit_basis <- fits[["model1"]]

save_plot <- function(p, filename, width = 7, height = 5, dpi = 600) {
  ggsave(
    filename = file.path(out_dir, filename),
    plot = p,
    width = width,
    height = height,
    dpi = dpi
  )
}

# -----

```

```

# 0) Basic diagnostics for key parameters
# -----
diag_tbl <- fit_basis$summary(variables = c("pi", "theta")) %>%
  dplyr::select(variable, rhat, ess_bulk, ess_tail)

diag_bad <- diag_tbl %>%
  dplyr::filter(is.finite(rhat), rhat > 1.01) %>%
  dplyr::arrange(desc(rhat))

diag_bad

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# Divergences, treedepth hits, if available (cmdstanr)
div_info <- tryCatch(fit_basis$diagnostic_summary(), error = function(e) NULL)
if (is.null(div_info)) {
  div_info <- tryCatch(fit_basis$cmdstan_diagnose(), error = function(e) NULL)
}
div_info

## $num_divergent
## [1] 0 0 0 0
##
## $num_max_treedepth
## [1] 0 0 0 0
##
## $ebfmi
## [1] 0.9495213 0.9361243 0.9325181 0.9066486

# -----
# 1) Confusion matrices (theta), posterior mean and uncertainty
# -----
theta_draws_mat <- fit_basis$draws("theta", format = "draws_matrix")
theta_mean_vec <- colMeans(theta_draws_mat)

# Expectation: theta has dimensions [J rater, K true, K given]
theta_mean_arr <- array(
  theta_mean_vec,
  dim = c(J, K, K),
  dimnames = list(
    rater = c("human1", "human2", "tool"),
    true = isco_levels,
    given = isco_levels
  )
)

theta_human1_mean <- theta_mean_arr["human1", , ]
theta_human2_mean <- theta_mean_arr["human2", , ]
theta_tool_mean <- theta_mean_arr["tool", , ]

theta_human1_mean

##      given
## true      1      2      3

```

```
##      1 0.91075729 0.07161341 0.01762930
##      2 0.01348360 0.94100368 0.04551273
##      3 0.05920051 0.05757161 0.88322789
```

```
rowSums(theta_human1_mean)
```

```
## 1 2 3
```

```
## 1 1 1
```

```
theta_human2_mean
```

```
##      given
## true      1      2      3
##      1 0.908573279 0.07313811 0.01828861
##      2 0.007151426 0.97904012 0.01380845
##      3 0.021194449 0.01788347 0.96092208
```

```
rowSums(theta_human2_mean)
```

```
## 1 2 3
```

```
## 1 1 1
```

```
theta_tool_mean
```

```
##      given
## true      1      2      3
##      1 0.76565145 0.1411890 0.09315953
##      2 0.03740508 0.9186500 0.04394493
##      3 0.06789798 0.1133579 0.81874409
```

```
rowSums(theta_tool_mean)
```

```
## 1 2 3
```

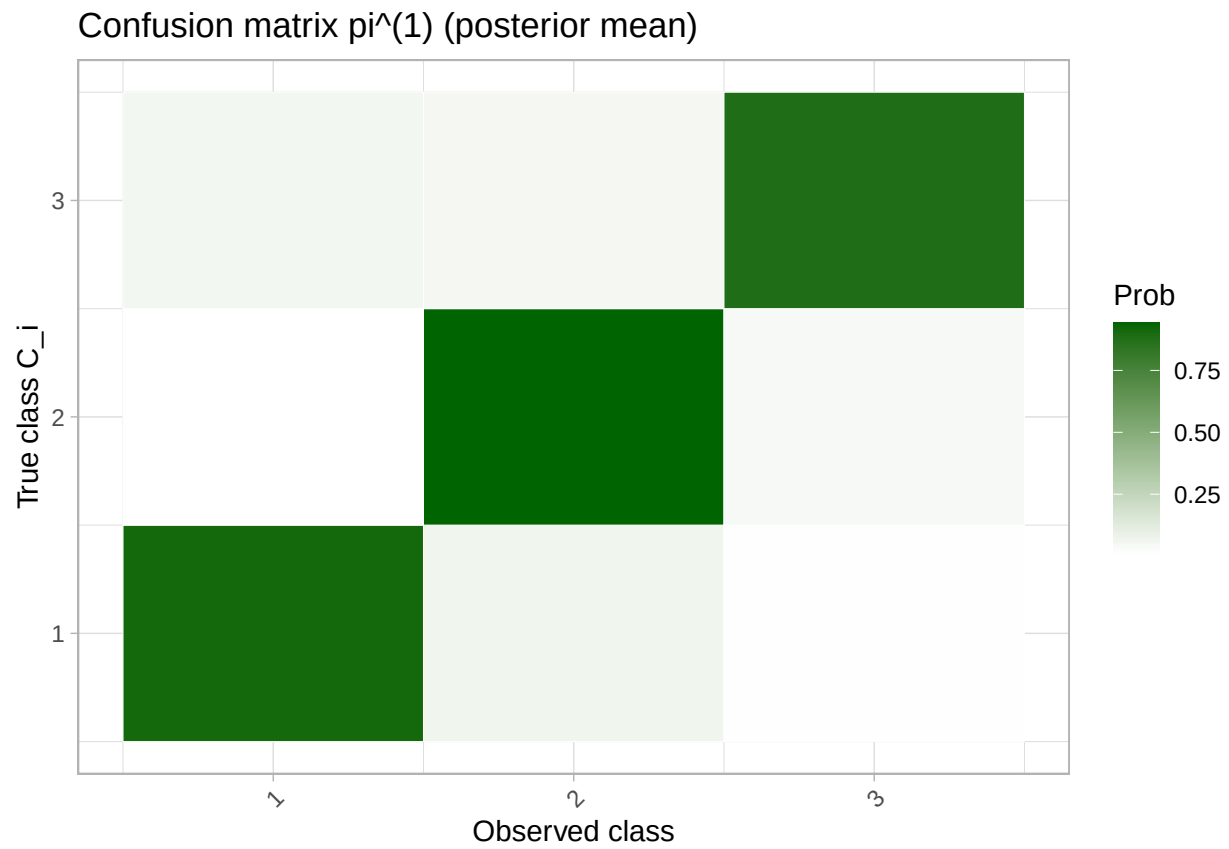
```
## 1 1 1
```

```
plot_theta <- function(mat, title_txt) {
  long <- reshape2::melt(mat, varnames = c("True", "Given"), value.name = "Prob")
  ggplot(long, aes(x = Given, y = True, fill = Prob)) +
    geom_tile(color = "white") +
    scale_fill_gradient(low = "white", high = "darkgreen") +
    theme_light() +
    labs(
      title = title_txt,
      x = "Observed class",
      y = "True class C_i"
    ) +
    theme(axis.text.x = element_text(angle = 45, hjust = 1))
}
```

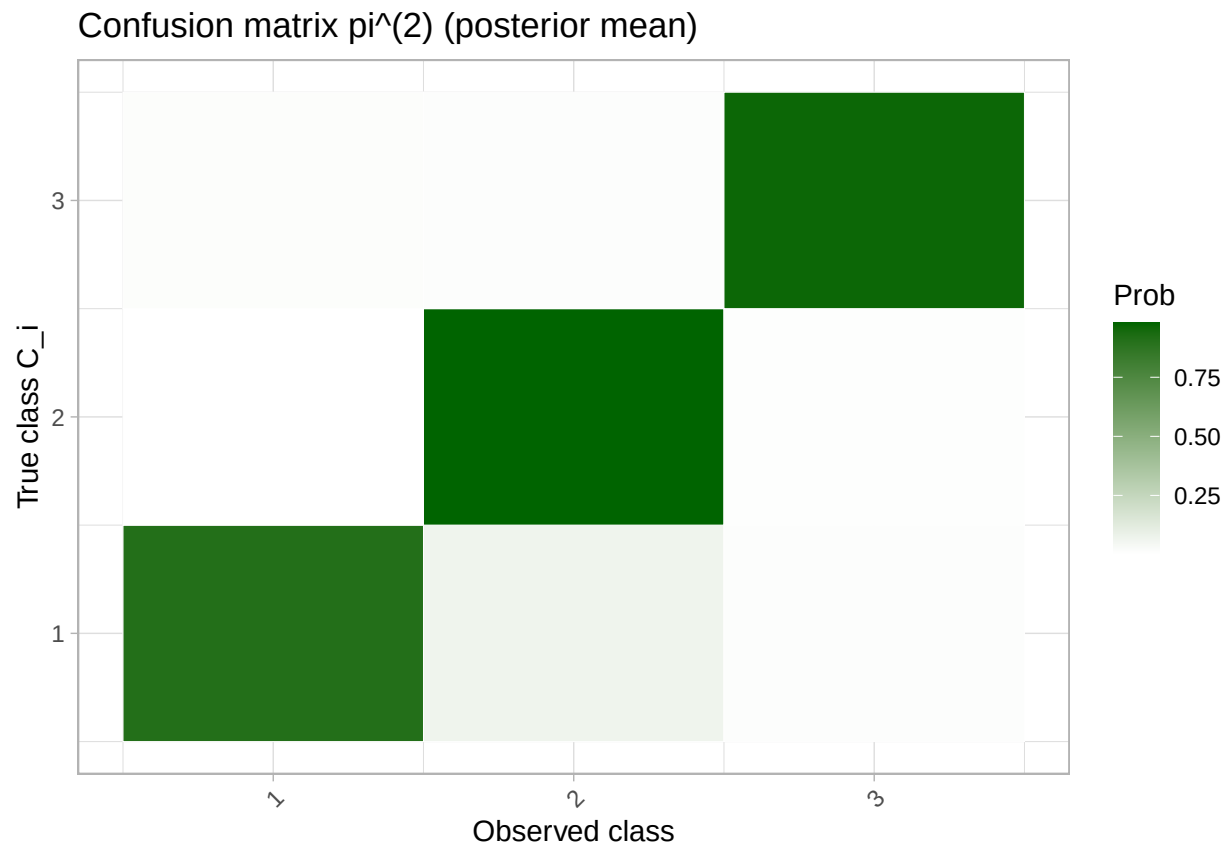
```
p_theta_h1_mean <- plot_theta(theta_human1_mean, "Confusion matrix  $\pi^{(1)}$  (posterior mean)")
p_theta_h2_mean <- plot_theta(theta_human2_mean, "Confusion matrix  $\pi^{(2)}$  (posterior mean)")
p_theta_tool_mean <- plot_theta(theta_tool_mean, "Confusion matrix  $\pi^{(3)}$  (posterior mean)")
```

```
save_plot(p_theta_h1_mean, "theta_human1_mean.png")
save_plot(p_theta_h2_mean, "theta_human2_mean.png")
save_plot(p_theta_tool_mean, "theta_tool_mean.png")
```

```
p_theta_h1_mean
```

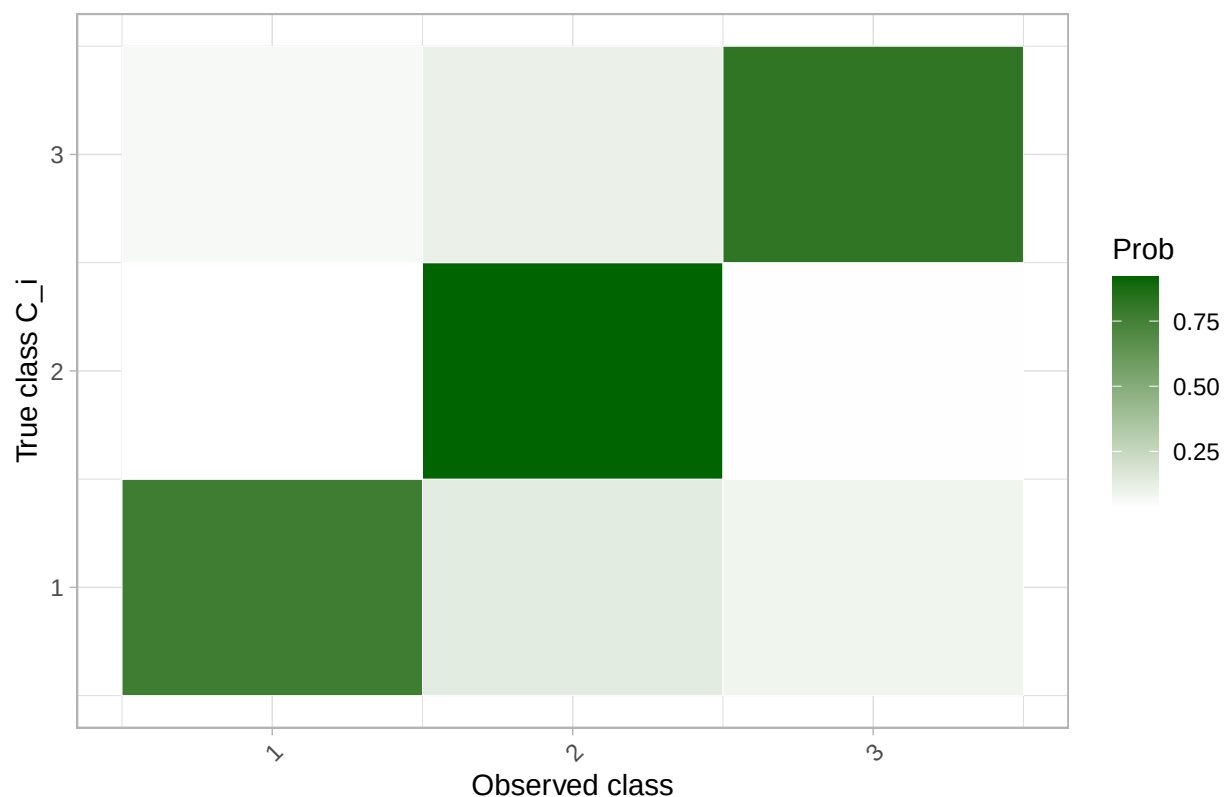


p_theta_h2_mean



p_theta_tool_mean

Confusion matrix $\pi^{(3)}$ (posterior mean)



```
# Credible intervals for all theta entries
theta_summ <- summarise_draws(
  as_draws_df(fit_basis$draws("theta")),
  mean, sd,
  ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
)
theta_summ
```

```
## # A tibble: 27 x 6
##   variable      mean      sd    '5%'    '50%'    '95%'
##   <chr>         <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 theta[1,1,1] 0.911 0.0335 0.851 0.914 0.959
## 2 theta[2,1,1] 0.909 0.0350 0.845 0.912 0.959
## 3 theta[3,1,1] 0.766 0.0471 0.686 0.767 0.840
## 4 theta[1,2,1] 0.0135 0.00802 0.00281 0.0122 0.0285
## 5 theta[2,2,1] 0.00715 0.00592 0.000539 0.00574 0.0186
## 6 theta[3,2,1] 0.0374 0.0117 0.0203 0.0362 0.0583
## 7 theta[1,3,1] 0.0592 0.0220 0.0275 0.0567 0.0992
## 8 theta[2,3,1] 0.0212 0.0145 0.00322 0.0184 0.0486
## 9 theta[3,3,1] 0.0679 0.0224 0.0353 0.0656 0.108
## 10 theta[1,1,2] 0.0716 0.0298 0.0294 0.0683 0.125
## # i 17 more rows
```

```
# Posterior SD heatmaps for theta (uncertainty by cell)
theta_sd_vec <- apply(theta_draws_mat, 2, sd)
theta_sd_arr <- array(
  theta_sd_vec,
```

```

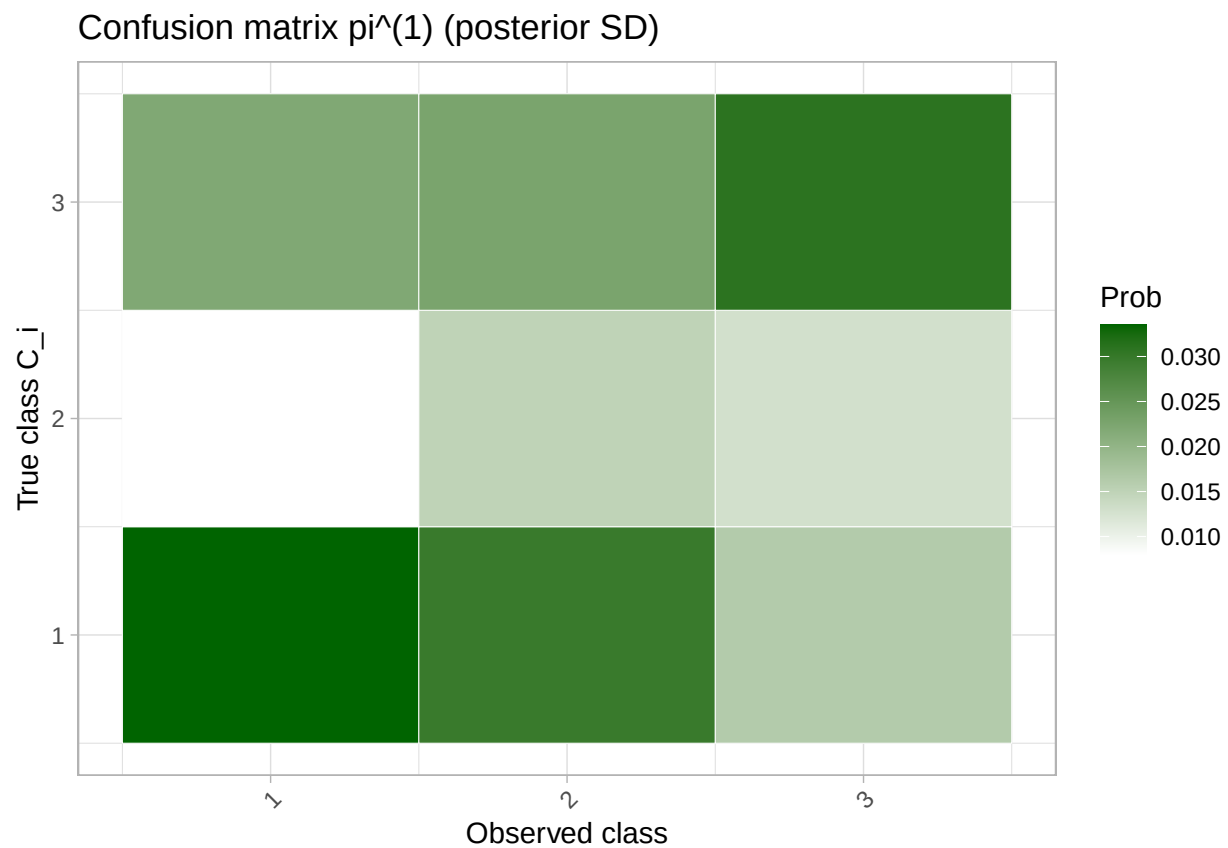
dim = c(J, K, K),
dimnames = list(
  rater = c("human1", "human2", "tool"),
  true = isco_levels,
  given = isco_levels
)
)

p_theta_h1_sd <- plot_theta(theta_sd_arr["human1", , ], "Confusion matrix  $\pi^{(1)}$  (posterior SD)")
p_theta_h2_sd <- plot_theta(theta_sd_arr["human2", , ], "Confusion matrix  $\pi^{(2)}$  (posterior SD)")
p_theta_tool_sd <- plot_theta(theta_sd_arr["tool", , ], "Confusion matrix  $\pi^{(3)}$  (posterior SD)")

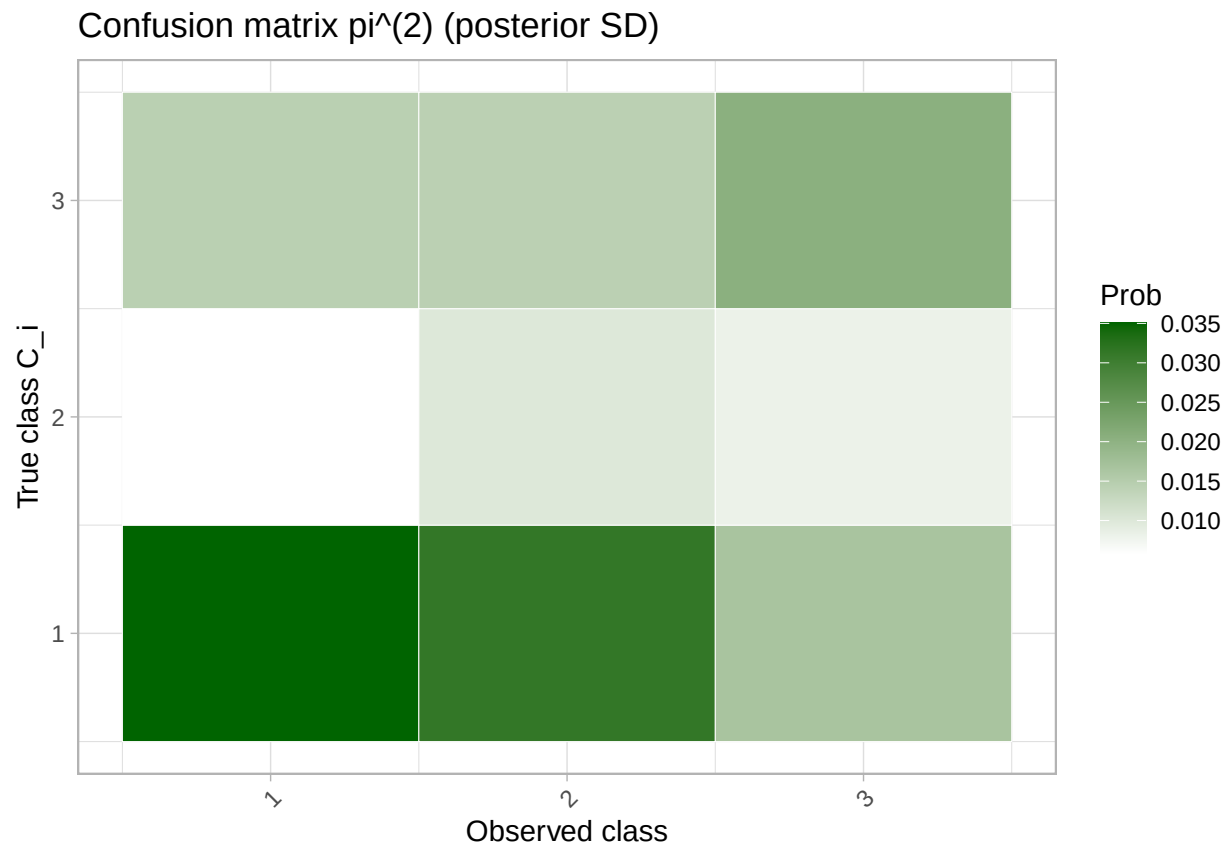
save_plot(p_theta_h1_sd, "theta_human1_sd.png")
save_plot(p_theta_h2_sd, "theta_human2_sd.png")
save_plot(p_theta_tool_sd, "theta_tool_sd.png")

p_theta_h1_sd

```

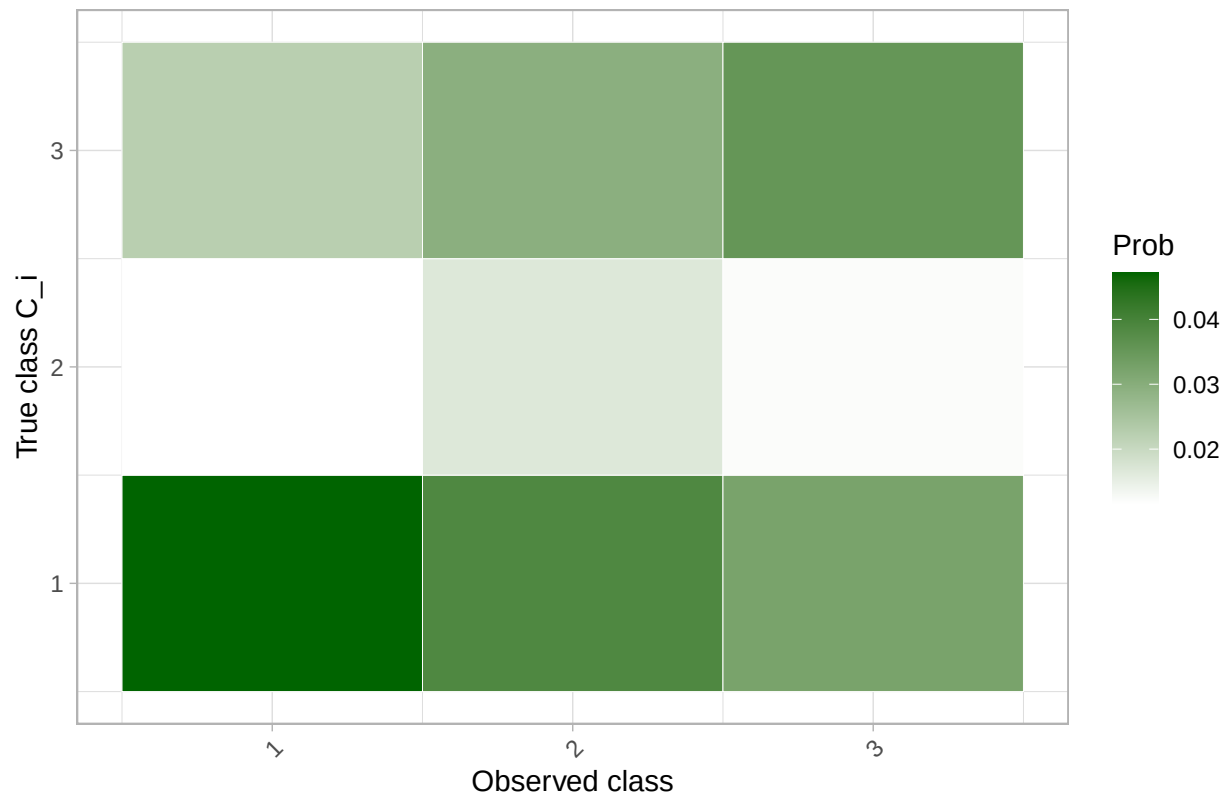


```
p_theta_h2_sd
```

p_theta_tool_sd

Confusion matrix $\pi^{(3)}$ (posterior SD)



```
# -----
# 2) Class prevalence ( $\pi$ ), posterior mean and uncertainty
# -----
pi_summ <- summarise_draws(
  as_draws_df(fit_basis$draws("pi")),
  mean, sd,
  ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
)
pi_summ
```

```
## # A tibble: 3 x 6
##   variable mean      sd `5%` `50%` `95%`
##   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 pi[1]      0.161 0.0176 0.132 0.160 0.191
## 2 pi[2]      0.592 0.0236 0.553 0.592 0.630
## 3 pi[3]      0.247 0.0208 0.214 0.247 0.282
```

```
pi_means <- colMeans(as_draws_matrix(fit_basis$draws("pi")))
pi_df <- data.frame(
  class = isco_levels,
  prevalence = as.numeric(pi_means)
)
```

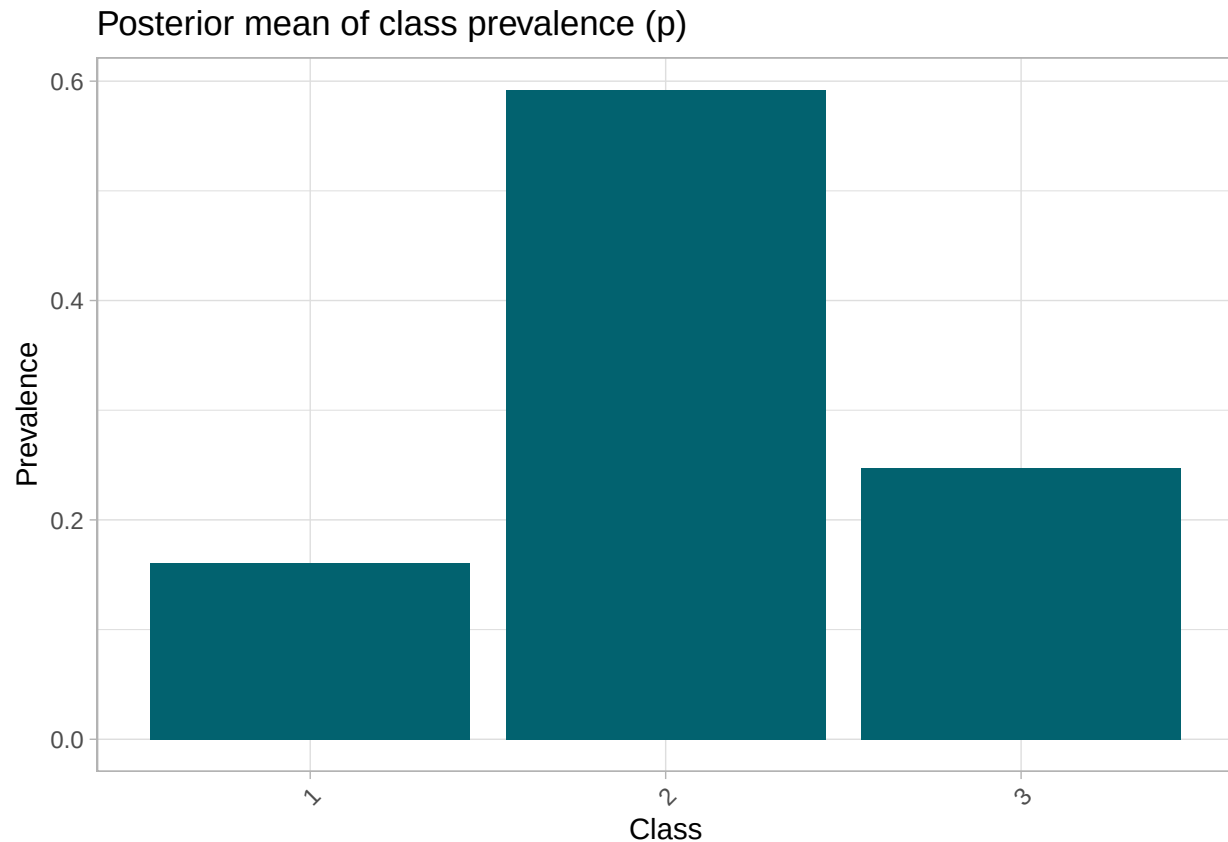
```
p_pi_bar <- ggplot(pi_df, aes(x = class, y = prevalence)) +
  geom_bar(stat = "identity", fill = "#02626f") +
  theme_light() +
  labs(
```

```

title = "Posterior mean of class prevalence (p)",
x = "Class",
y = "Prevalence"
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

save_plot(p_pi_bar, "pi_mean_bar.png", width = 6, height = 4)
p_pi_bar

```



```

# -----
# 2b) Rater level scalar summaries from theta
# -----
pi_mean_vec <- as.numeric(pi_means)
names(pi_mean_vec) <- isco_levels

weighted_diag_mass <- function(mat, w) sum(diag(mat) * w)
weighted_offdiag_mass <- function(mat, w) sum((1 - diag(mat)) * w)

rater_summary <- data.frame(
  rater = c("human1", "human2", "tool"),
  weighted_diag_mass = c(
    weighted_diag_mass(theta_human1_mean, pi_mean_vec),
    weighted_diag_mass(theta_human2_mean, pi_mean_vec),
    weighted_diag_mass(theta_tool_mean, pi_mean_vec)
  ),
  weighted_offdiag_mass = c(

```

```

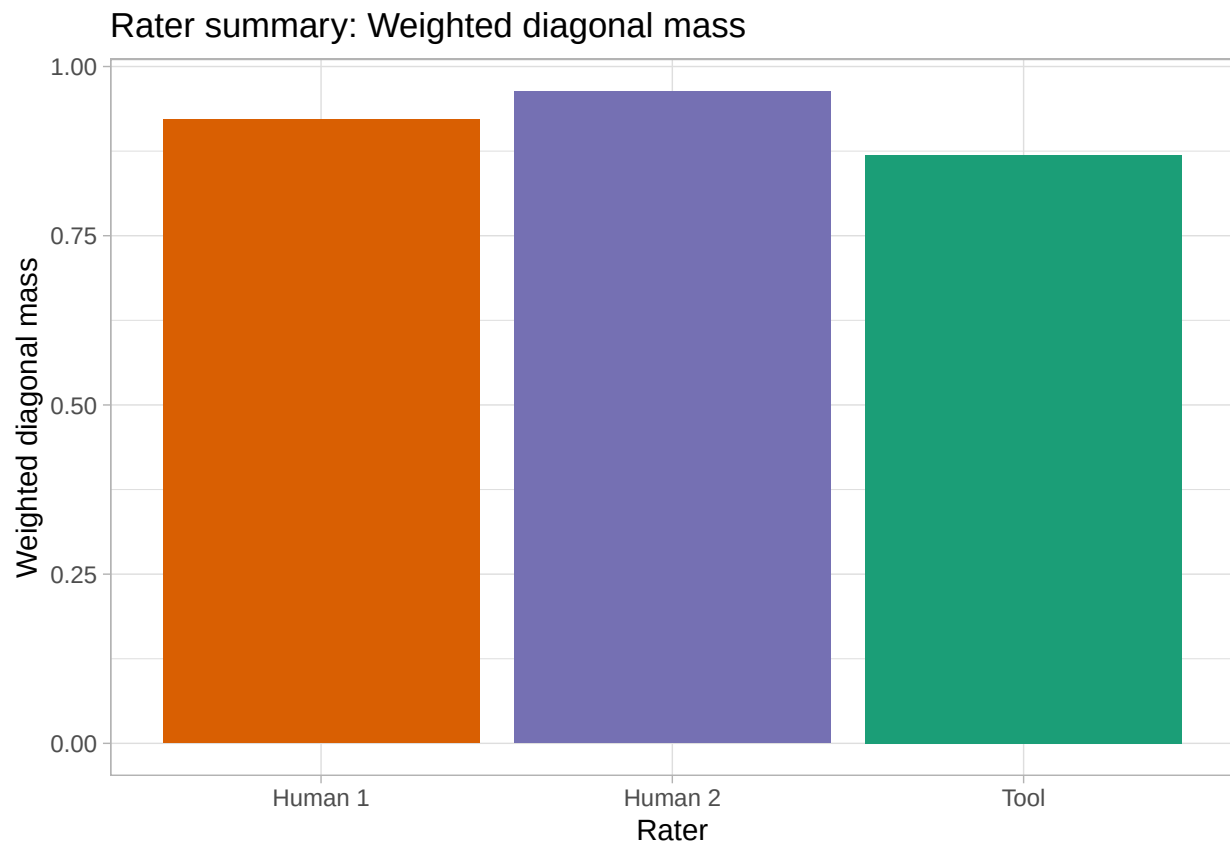
    weighted_offdiag_mass(theta_human1_mean, pi_mean_vec),
    weighted_offdiag_mass(theta_human2_mean, pi_mean_vec),
    weighted_offdiag_mass(theta_tool_mean, pi_mean_vec)
  )
)
rater_summary

##      rater weighted_diag_mass weighted_offdiag_mass
## 1 human1      0.9218564      0.07814360
## 2 human2      0.9632392      0.03676081
## 3  tool      0.8693640      0.13063599

p_rater_diag <- ggplot(rater_summary, aes(x = rater, y = weighted_diag_mass, fill = rater)) +
  geom_bar(stat = "identity") +
  scale_x_discrete(labels = c("human1" = "Human 1", "human2" = "Human 2", "tool" = "Tool")) +
  scale_fill_manual(values = c("human1" = "#d95f02", "human2" = "#7570b3", "tool" = "#1b9e77"),
    guide = "none") +
  theme_light() +
  labs(
    title = "Rater summary: Weighted diagonal mass",
    x = "Rater",
    y = "Weighted diagonal mass"
  )

save_plot(p_rater_diag, "rater_weighted_diag_mass.png", width = 6, height = 4)
p_rater_diag

```



```

# -----
# 3) Posterior class probabilities per item ( $q_{i,j} = P(C_i = j \mid \text{data})$ )
# -----
qz_draws_mat <- fit_basis$draws("q_z", format = "draws_matrix")
qz_mean_vec <- colMeans(qz_draws_mat)

qz_mean <- matrix(
  qz_mean_vec,
  nrow = I, ncol = K,
  dimnames = list(item = item_ids, class = isco_levels)
)

qz_df <- as.data.frame(qz_mean)
p_cols <- paste0("p_", isco_levels)
colnames(qz_df) <- p_cols
qz_df$item_id <- item_ids

# MAP label
qz_df$map_class_idx <- max.col(qz_df[p_cols], ties.method = "first")
qz_df$map_class_label <- isco_levels[qz_df$map_class_idx]

# Uncertainty measures
eps <- 1e-12
qz_df$max_p <- apply(qz_df[p_cols], 1, max)

qz_df$entropy <- -rowSums(as.matrix(qz_df[p_cols]) * log(as.matrix(qz_df[p_cols]) + eps))

qz_df$margin <- apply(qz_df[p_cols], 1, function(p) {
  ps <- sort(p, decreasing = TRUE)
  ps[1] - ps[2]
})

# Credible set per item, smallest set reaching cumulative posterior mass >= level
credible_set <- function(p, classes, level = 0.95) {
  o <- order(p, decreasing = TRUE)
  cum <- cumsum(p[o])
  m <- which(cum >= level)[1]
  classes[o[seq_len(m)]]
}
qz_df$cs95 <- apply(qz_df[p_cols], 1, function(x) credible_set(x, isco_levels, level = 0.95))

table(lengths(qz_df$cs95))

##
## 1 2 3
## 444 9 7

# Plots of uncertainty and confidence
p_maxp_hist <- ggplot(qz_df, aes(x = max_p)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior max probability", y = "Count",
       title = "Model confidence per item") +
  xlim(0, 1.1)

```

```

p_entropy_hist <- ggplot(qz_df, aes(x = entropy)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior entropy", y = "Count",
       title = "Item uncertainty (entropy)") +
  xlim(-0.1, 1.0)

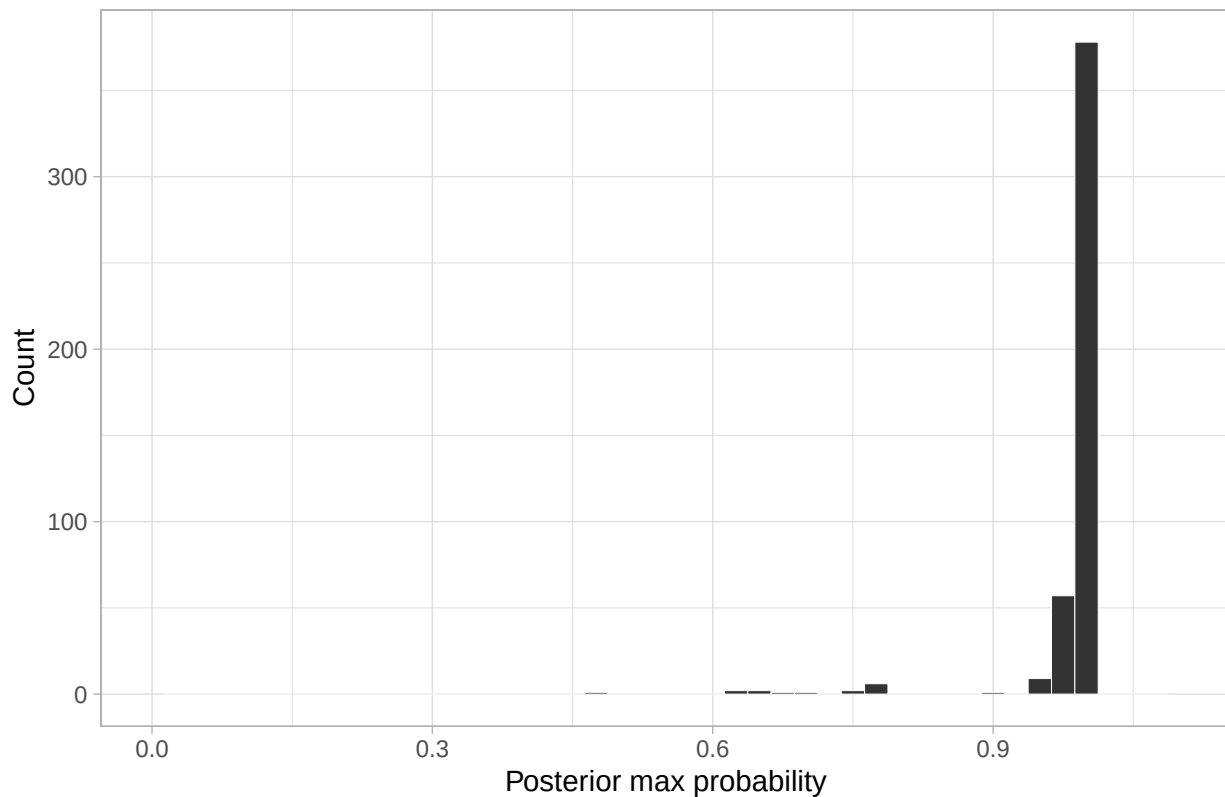
p_margin_hist <- ggplot(qz_df, aes(x = margin)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior margin (top1 minus top2)", y = "Count",
       title = "Item uncertainty (margin)") +
  xlim(0, 1.1)

save_plot(p_maxp_hist, "qz_maxp_hist.png", width = 6, height = 4)
save_plot(p_entropy_hist, "qz_entropy_hist.png", width = 6, height = 4)
save_plot(p_margin_hist, "qz_margin_hist.png", width = 6, height = 4)

p_maxp_hist

```

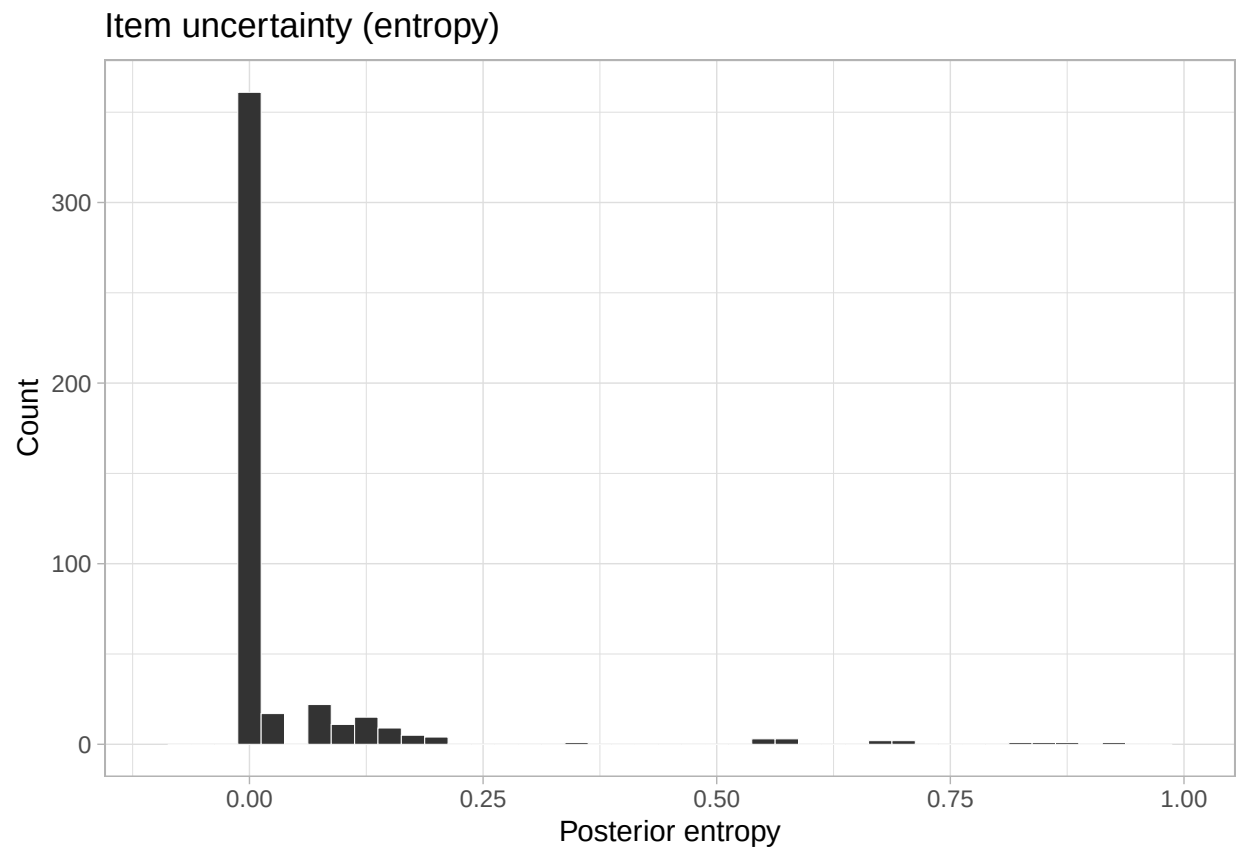
Model confidence per item



```

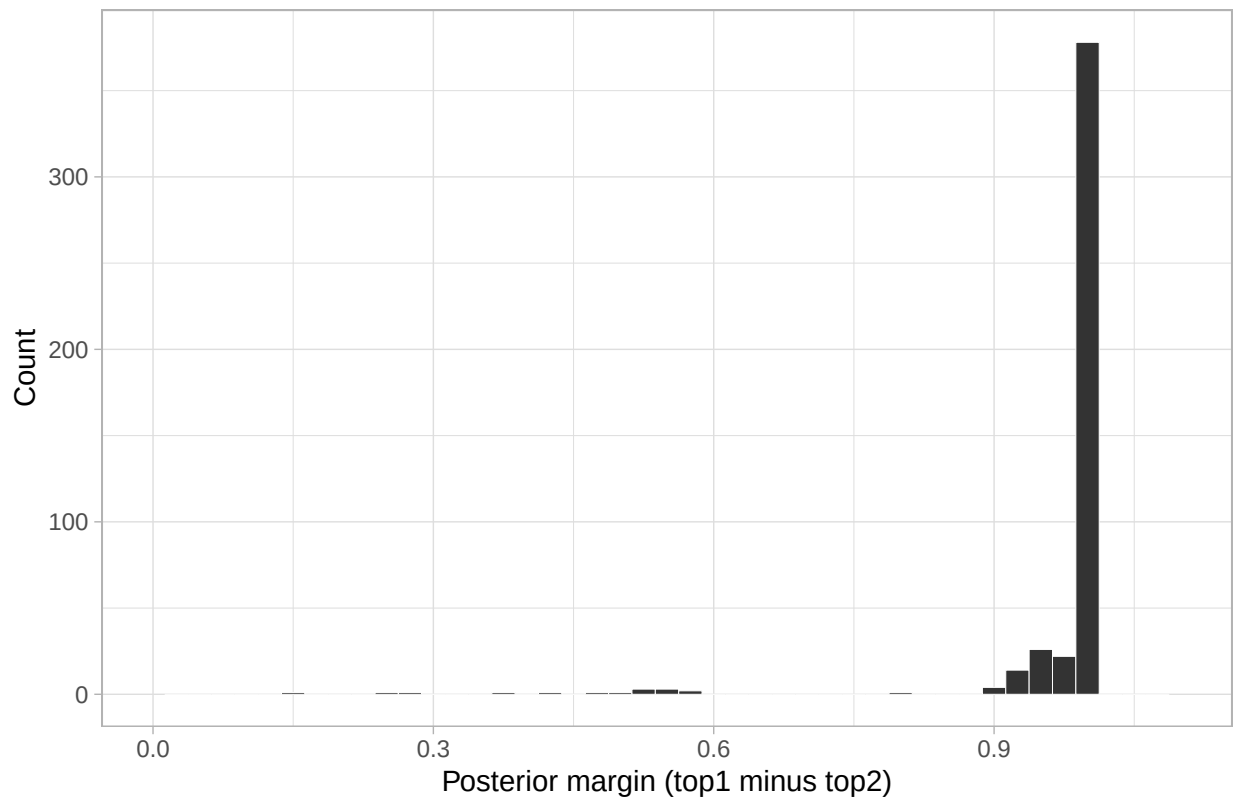
p_entropy_hist

```



p_margin_hist

Item uncertainty (margin)



```
# -----
# 3b) MAP distribution vs pi (sanity check)
# -----
map_tab <- qz_df %>%
  count(map_class_label, name = "n_map") %>%
  mutate(map_share = n_map / sum(n_map)) %>%
  rename(class = map_class_label)

pi_tab <- data.frame(class = isco_levels, pi_mean = as.numeric(pi_means))

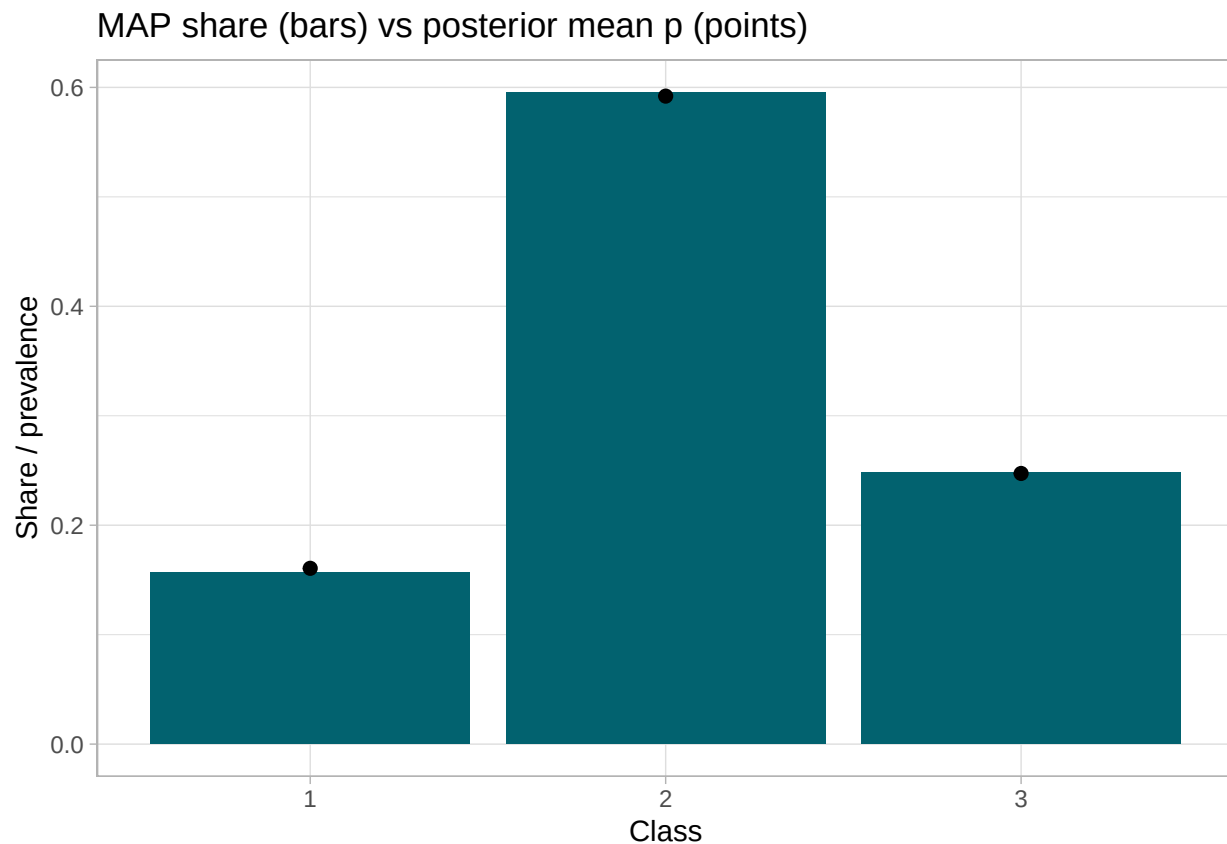
map_vs_pi <- left_join(map_tab, pi_tab, by = "class")
map_vs_pi
```

```
##   class n_map map_share  pi_mean
## 1     1     72 0.1565217 0.1606462
## 2     2    274 0.5956522 0.5920477
## 3     3    114 0.2478261 0.2473061
```

```
p_map_vs_pi <- ggplot(map_vs_pi, aes(x = class)) +
  geom_col(aes(y = map_share), fill = "#02626f") +
  geom_point(aes(y = pi_mean), size = 2) +
  theme_light() +
  labs(
    title = "MAP share (bars) vs posterior mean p (points)",
    x = "Class",
    y = "Share / prevalence"
  )
```



```
save_plot(p_map_vs_pi, "map_share_vs_pi.png", width = 6, height = 4)
p_map_vs_pi
```



```
# -----
# 4) Merge predictions with original ratings, include uncertainty
# -----
ratings <- isco_data %>%
  dplyr::select(item_id, isco_human1, isco_human2, isco_tool)

predictions <- qz_df %>%
  dplyr::select(item_id, map_class_label, all_of(p_cols), max_p, entropy, margin, cs95)

ratings_with_predictions <- ratings %>%
  dplyr::left_join(predictions, by = "item_id") %>%
  dplyr::rename(
    predicted_class = map_class_label,
    rating_human1 = isco_human1,
    rating_human2 = isco_human2,
    rating_tool = isco_tool
  ) %>%
  dplyr::arrange(item_id)

head(ratings_with_predictions, 30)
```

```
##      item_id rating_human1 rating_human2 rating_tool predicted_class      p_1      p_2
##      <int>      <char>      <char>      <char>      <char>      <num>      <num>
```

## 1:	1	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 2:	2	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 3:	3	3	1	2	3	3.287328e-01	1.854526e-01	4.8581
## 4:	4	2	2	3	2	3.622564e-03	9.867354e-01	9.6420
## 5:	5	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 6:	6	1	1	1	1	9.997651e-01	2.192146e-05	2.1295
## 7:	7	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 8:	8	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 9:	9	2	2	3	2	3.622564e-03	9.867354e-01	9.6420
## 10:	10	3	3	3	3	2.994898e-05	9.694723e-05	9.9987
## 11:	11	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 12:	12	3	3	1	3	3.230342e-03	1.134353e-03	9.9563
## 13:	13	2	2	1	2	3.452029e-02	9.645274e-01	9.5228
## 14:	14	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 15:	15	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 16:	16	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 17:	17	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 18:	18	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 19:	19	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 20:	20	3	3	3	3	2.994898e-05	9.694723e-05	9.9987
## 21:	21	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 22:	22	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 23:	23	1	1	1	1	9.997651e-01	2.192146e-05	2.1295
## 24:	24	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 25:	25	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 26:	26	2	2	1	2	3.452029e-02	9.645274e-01	9.5228
## 27:	27	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 28:	28	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 29:	29	2	2	2	2	2.438711e-04	9.996963e-01	5.9851
## 30:	30	3	3	2	3	3.604717e-04	1.569480e-02	9.8394
##	item_id	rating_human1	rating_human2	rating_tool	predicted_class	p_1	p_2	
##	max_p	entropy	margin	cs95				
##	<num>	<num>	<num>	<list>				
## 1:	0.9996963	0.002914379	0.9994524	2				
## 2:	0.9996963	0.002914379	0.9994524	2				
## 3:	0.4858146	1.028921278	0.1570818	3,1,2				
## 4:	0.9867354	0.078292030	0.9770933	2				
## 5:	0.9996963	0.002914379	0.9994524	2				
## 6:	0.9997651	0.002270446	0.9995522	1				
## 7:	0.9996963	0.002914379	0.9994524	2				
## 8:	0.9996963	0.002914379	0.9994524	2				
## 9:	0.9867354	0.078292030	0.9770933	2				
## 10:	0.9998731	0.001334760	0.9997762	3				
## 11:	0.9996963	0.002914379	0.9994524	2				
## 12:	0.9956353	0.030574541	0.9924050	3				
## 13:	0.9645274	0.157663014	0.9300071	2				
## 14:	0.9996963	0.002914379	0.9994524	2				
## 15:	0.9996963	0.002914379	0.9994524	2				
## 16:	0.9996963	0.002914379	0.9994524	2				
## 17:	0.9996963	0.002914379	0.9994524	2				
## 18:	0.9996963	0.002914379	0.9994524	2				
## 19:	0.9996963	0.002914379	0.9994524	2				
## 20:	0.9998731	0.001334760	0.9997762	3				
## 21:	0.9996963	0.002914379	0.9994524	2				

```
## 22: 0.9996963 0.002914379 0.9994524      2
## 23: 0.9997651 0.002270446 0.9995522      1
## 24: 0.9996963 0.002914379 0.9994524      2
## 25: 0.9996963 0.002914379 0.9994524      2
## 26: 0.9645274 0.157663014 0.9300071      2
## 27: 0.9996963 0.002914379 0.9994524      2
## 28: 0.9996963 0.002914379 0.9994524      2
## 29: 0.9996963 0.002914379 0.9994524      2
## 30: 0.9839447 0.083986429 0.9682499      3
##      max_p      entropy      margin      cs95
```

```
# Most uncertain items by entropy, include posterior probabilities
```

```
most_uncertain <- ratings_with_predictions %>%
```

```
  dplyr::arrange(desc(entropy)) %>%
```

```
  dplyr::select(item_id, predicted_class, entropy, margin, max_p, all_of(p_cols),
                rating_human1, rating_human2, rating_tool)
```

```
head(most_uncertain, 20)
```

```
##      item_id predicted_class  entropy      margin      max_p      p_1      p_2      p_3 rating_l
##      <int>      <char>      <num>      <num>      <num>      <num>      <num>      <num>
## 1:         3              3 1.0289213 0.1570818 0.4858146 0.328732785 0.18545263 0.485814590
## 2:        388              1 0.9209594 0.3768367 0.6157392 0.615739211 0.23890251 0.145358281
## 3:         94              3 0.8671926 0.4315979 0.6554317 0.120734525 0.22383377 0.655431706
## 4:        129              1 0.8557463 0.4827926 0.6722481 0.672248113 0.13829642 0.189455465
## 5:        165              2 0.8132061 0.5101016 0.6978115 0.114478659 0.69781146 0.187709879
## 6:        341              2 0.7090817 0.2505658 0.6204219 0.369856059 0.62042191 0.009722033
## 7:        365              3 0.6878467 0.2819231 0.6376476 0.006627817 0.35572457 0.637647618
## 8:        327              3 0.6842555 0.5734191 0.7584550 0.185035958 0.05650902 0.758455018
## 9:        368              3 0.6842555 0.5734191 0.7584550 0.185035958 0.05650902 0.758455018
## 10:       177              2 0.5673383 0.5356863 0.7655327 0.004620922 0.76553267 0.229846409
## 11:       301              2 0.5673383 0.5356863 0.7655327 0.004620922 0.76553267 0.229846409
## 12:       394              2 0.5673383 0.5356863 0.7655327 0.004620922 0.76553267 0.229846409
## 13:       183              2 0.5573459 0.5514921 0.7734695 0.221977463 0.77346953 0.004553006
## 14:       260              2 0.5573459 0.5514921 0.7734695 0.221977463 0.77346953 0.004553006
## 15:       452              2 0.5573459 0.5514921 0.7734695 0.221977463 0.77346953 0.004553006
## 16:       421              3 0.3484203 0.7972137 0.8967203 0.099506619 0.00377304 0.896720341
## 17:       195              1 0.2019412 0.9074088 0.9523492 0.952349208 0.04494037 0.002710424
## 18:       214              1 0.2019412 0.9074088 0.9523492 0.952349208 0.04494037 0.002710424
## 19:       359              1 0.2019412 0.9074088 0.9523492 0.952349208 0.04494037 0.002710424
## 20:       415              1 0.2019412 0.9074088 0.9523492 0.952349208 0.04494037 0.002710424
##      item_id predicted_class  entropy      margin      max_p      p_1      p_2      p_3 rating_l
##      rating_human2 rating_tool
##      <char>      <char>
## 1:         1          2
## 2:         2          3
## 3:         3          1
## 4:         1          3
## 5:         2          1
## 6:         1          2
## 7:         2          3
## 8:         3          2
## 9:         3          2
## 10:        3          2
## 11:        3          2
```

```
## 12:      3      2
## 13:      2      2
## 14:      2      2
## 15:      2      2
## 16:      1      3
## 17:      2      1
## 18:      2      1
## 19:      2      1
## 20:      2      1
##      rating_human2 rating_tool
```

```
# -----
# 5) Observed agreement, descriptive, not model based
# -----
mean(isco_data$isco_human1 == isco_data$isco_tool, na.rm = TRUE)
```

```
## [1] 0.8065217
```

```
mean(isco_data$isco_human2 == isco_data$isco_tool, na.rm = TRUE)
```

```
## [1] 0.8456522
```

```
isco_data %>%
  dplyr::count(isco_human1, isco_human2, isco_tool) %>%
  dplyr::arrange(desc(n))
```

```
##      isco_human1 isco_human2 isco_tool      n
##      <char>      <char>      <char> <int>
## 1:      2      2      2      235
## 2:      3      3      3      80
## 3:      1      1      1      46
## 4:      2      2      3      11
## 5:      3      2      2      11
## 6:      3      3      2      11
## 7:      1      1      2      10
## 8:      2      2      1      9
## 9:      3      3      1      7
## 10:     1      1      3      6
## 11:     1      3      3      5
## 12:     2      3      3      5
## 13:     1      2      1      4
## 14:     2      1      1      4
## 15:     1      2      2      3
## 16:     2      3      2      3
## 17:     1      3      2      2
## 18:     1      2      3      1
## 19:     2      1      2      1
## 20:     2      1      3      1
## 21:     2      3      1      1
## 22:     3      1      2      1
## 23:     3      1      3      1
## 24:     3      2      1      1
## 25:     3      2      3      1
##      isco_human1 isco_human2 isco_tool      n
```

```
library(dplyr)
library(posterior)
```

```

library(ggplot2)
library(reshape2)

out_dir <- "img/model_results/isco/three_categories/alpha_informative"
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)

# use first fit
fit_basis <- fits[["model2"]]

save_plot <- function(p, filename, width = 7, height = 5, dpi = 600) {
  ggsave(
    filename = file.path(out_dir, filename),
    plot = p,
    width = width,
    height = height,
    dpi = dpi
  )
}

# -----
# 0) Basic diagnostics for key parameters
# -----
diag_tbl <- fit_basis$summary(variables = c("pi", "theta")) %>%
  dplyr::select(variable, rhat, ess_bulk, ess_tail)

diag_bad <- diag_tbl %>%
  dplyr::filter(is.finite(rhat), rhat > 1.01) %>%
  dplyr::arrange(desc(rhat))

diag_bad

## # A tibble: 0 x 4
## # i 4 variables: variable <chr>, rhat <dbl>, ess_bulk <dbl>, ess_tail <dbl>

# Divergences, treedepth hits, if available (cmdstanr)
div_info <- tryCatch(fit_basis$diagnostic_summary(), error = function(e) NULL)
if (is.null(div_info)) {
  div_info <- tryCatch(fit_basis$cmdstan_diagnose(), error = function(e) NULL)
}
div_info

## $num_divergent
## [1] 0 0 0 0
##
## $num_max_treedepth
## [1] 0 0 0 0
##
## $ebfmi
## [1] 0.9086901 0.9559382 0.9398511 0.9236251

# -----
# 1) Confusion matrices (theta), posterior mean and uncertainty
# -----
theta_draws_mat <- fit_basis$draws("theta", format = "draws_matrix")
theta_mean_vec <- colMeans(theta_draws_mat)

```

```

# Expectation: theta has dimensions [J rater, K true, K given]
theta_mean_arr <- array(
  theta_mean_vec,
  dim = c(J, K, K),
  dimnames = list(
    rater = c("human1", "human2", "tool"),
    true = isco_levels,
    given = isco_levels
  )
)

theta_human1_mean <- theta_mean_arr["human1", , ]
theta_human2_mean <- theta_mean_arr["human2", , ]
theta_tool_mean <- theta_mean_arr["tool", , ]

theta_human1_mean

##      given
## true      1      2      3
## 1 0.91140441 0.07149895 0.01709664
## 2 0.01334223 0.94158963 0.04506814
## 3 0.05928943 0.05821279 0.88249778

rowSums(theta_human1_mean)

## 1 2 3
## 1 1 1

theta_human2_mean

##      given
## true      1      2      3
## 1 0.908423344 0.07351964 0.01805702
## 2 0.007113708 0.97940826 0.01347804
## 3 0.021564918 0.01843963 0.95999545

rowSums(theta_human2_mean)

## 1 2 3
## 1 1 1

theta_tool_mean

##      given
## true      1      2      3
## 1 0.76647684 0.1407669 0.09275624
## 2 0.03725946 0.9188653 0.04387520
## 3 0.06807312 0.1142129 0.81771401

rowSums(theta_tool_mean)

## 1 2 3
## 1 1 1

plot_theta <- function(mat, title_txt) {
  long <- reshape2::melt(mat, varnames = c("True", "Given"), value.name = "Prob")
  ggplot(long, aes(x = Given, y = True, fill = Prob)) +
    geom_tile(color = "white") +

```

```

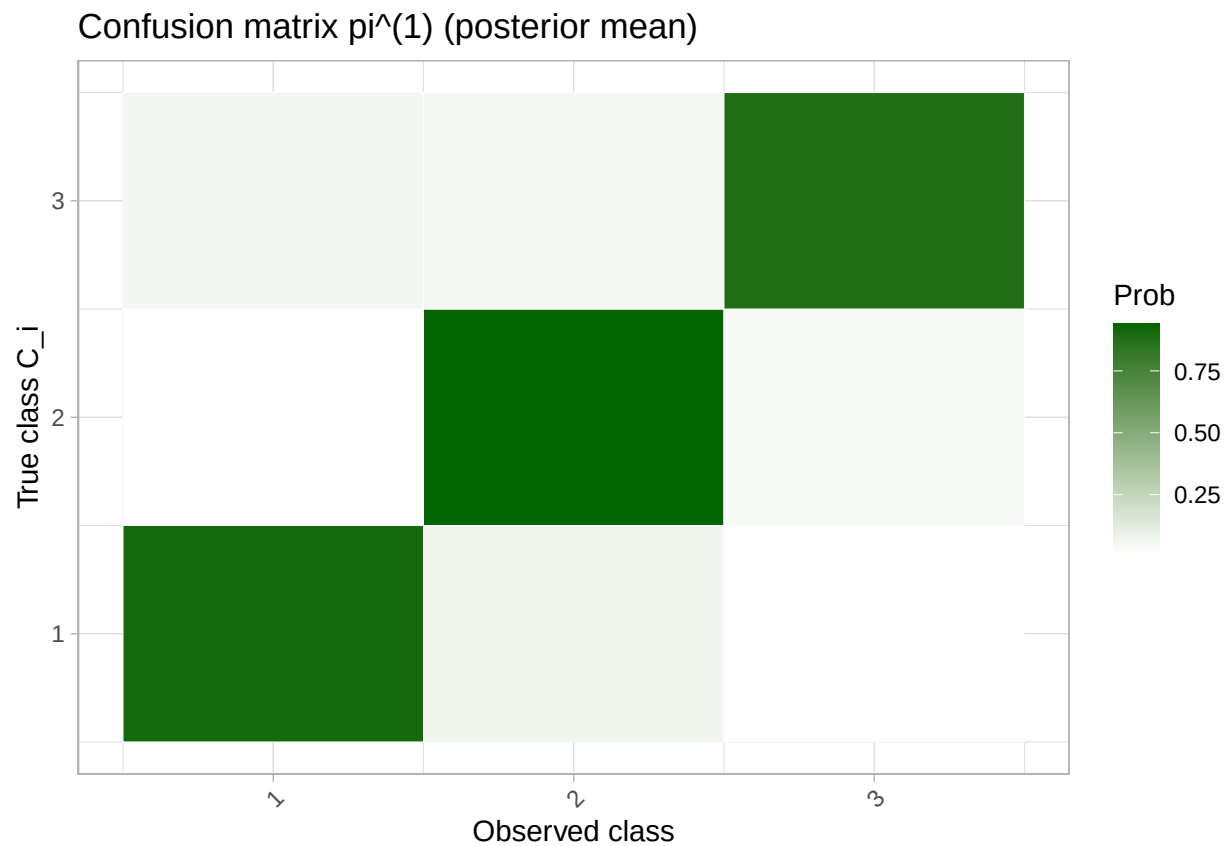
scale_fill_gradient(low = "white", high = "darkgreen") +
theme_light() +
labs(
  title = title_txt,
  x = "Observed class",
  y = "True class C_i"
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))
}

p_theta_h1_mean <- plot_theta(theta_human1_mean, "Confusion matrix pi^(1) (posterior mean)")
p_theta_h2_mean <- plot_theta(theta_human2_mean, "Confusion matrix pi^(2) (posterior mean)")
p_theta_tool_mean <- plot_theta(theta_tool_mean, "Confusion matrix pi^(3) (posterior mean)")

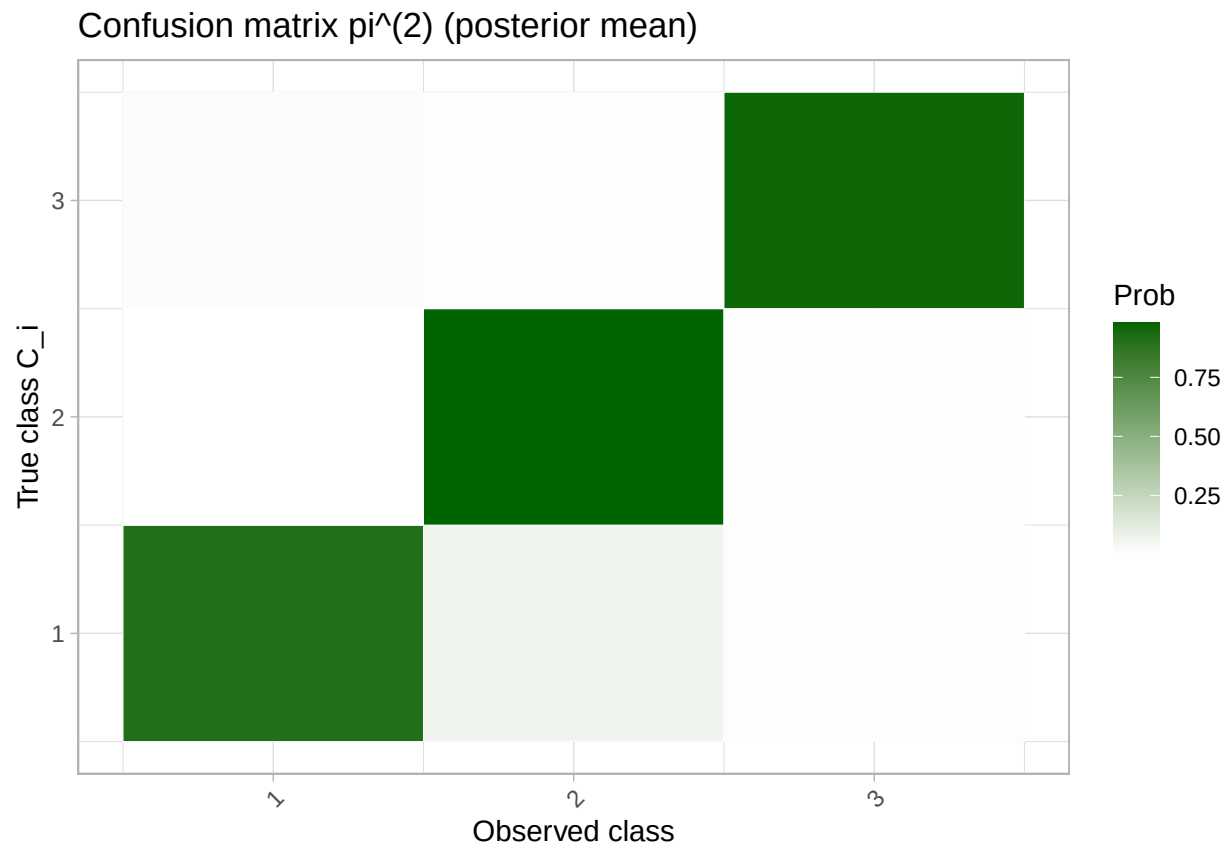
save_plot(p_theta_h1_mean, "theta_human1_mean.png")
save_plot(p_theta_h2_mean, "theta_human2_mean.png")
save_plot(p_theta_tool_mean, "theta_tool_mean.png")

p_theta_h1_mean

```

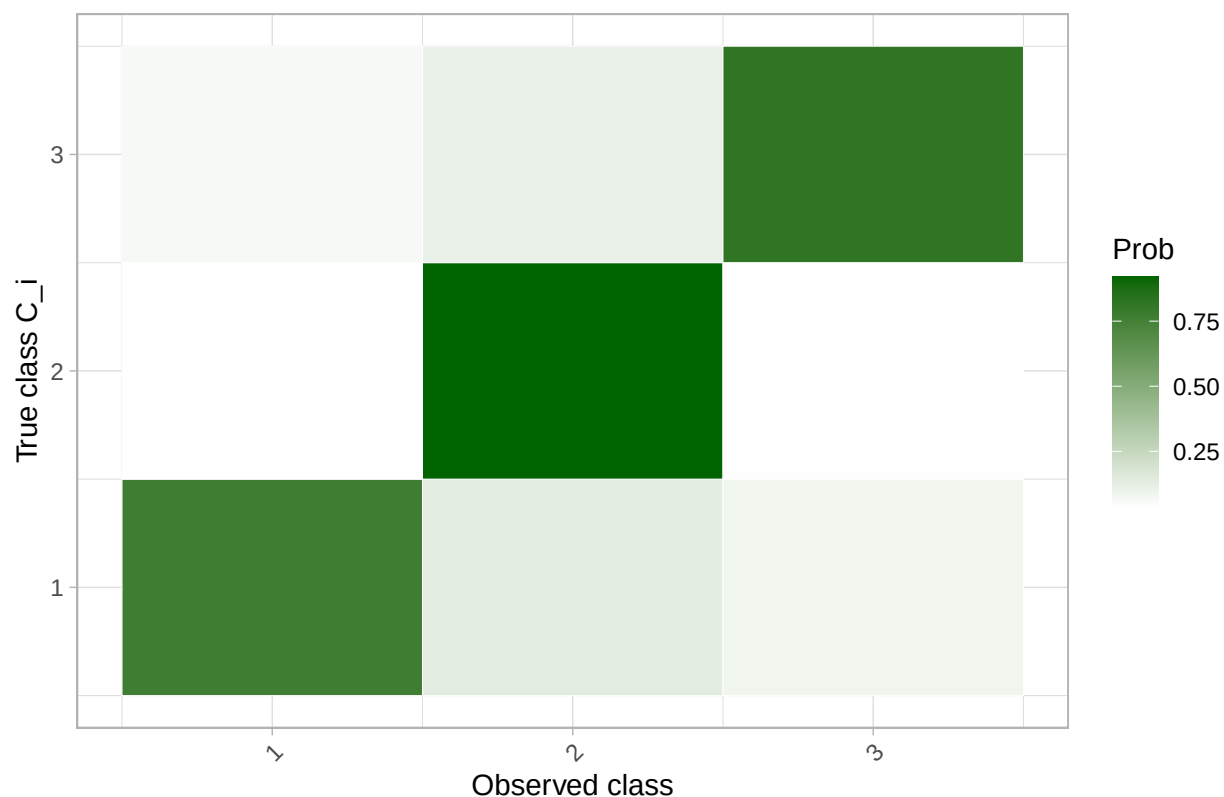


```
p_theta_h2_mean
```



p_theta_tool_mean

Confusion matrix $\pi^{(3)}$ (posterior mean)



```
# Credible intervals for all theta entries
theta_summ <- summarise_draws(
  as_draws_df(fit_basis$draws("theta")),
  mean, sd,
  ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
)
theta_summ
```

```
## # A tibble: 27 x 6
##   variable      mean      sd    '5%'    '50%'    '95%'
##   <chr>         <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 theta[1,1,1] 0.911  0.0333 0.852   0.915   0.960
## 2 theta[2,1,1] 0.908  0.0355 0.844   0.912   0.960
## 3 theta[3,1,1] 0.766  0.0466 0.687   0.769   0.839
## 4 theta[1,2,1] 0.0133 0.00814 0.00255 0.0120 0.0287
## 5 theta[2,2,1] 0.00711 0.00593 0.000517 0.00562 0.0188
## 6 theta[3,2,1] 0.0373 0.0119 0.0199 0.0361 0.0583
## 7 theta[1,3,1] 0.0593 0.0220 0.0274 0.0571 0.0985
## 8 theta[2,3,1] 0.0216 0.0145 0.00372 0.0187 0.0495
## 9 theta[3,3,1] 0.0681 0.0223 0.0355 0.0660 0.108
## 10 theta[1,1,2] 0.0715 0.0300 0.0291 0.0678 0.126
## # i 17 more rows
```

```
# Posterior SD heatmaps for theta (uncertainty by cell)
theta_sd_vec <- apply(theta_draws_mat, 2, sd)
theta_sd_arr <- array(
  theta_sd_vec,
```

```

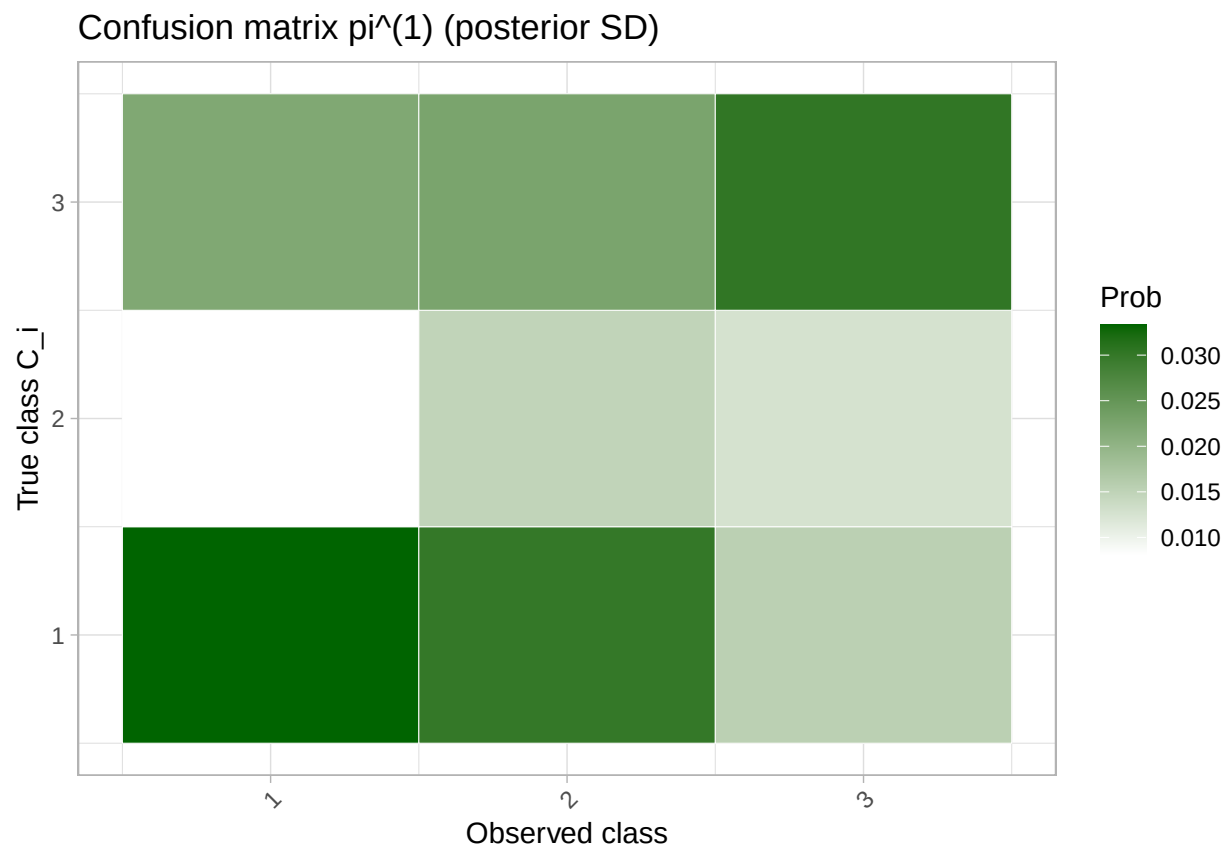
dim = c(J, K, K),
dimnames = list(
  rater = c("human1", "human2", "tool"),
  true = isco_levels,
  given = isco_levels
)
)

p_theta_h1_sd <- plot_theta(theta_sd_arr["human1", , ], "Confusion matrix  $\pi^{(1)}$  (posterior SD)")
p_theta_h2_sd <- plot_theta(theta_sd_arr["human2", , ], "Confusion matrix  $\pi^{(2)}$  (posterior SD)")
p_theta_tool_sd <- plot_theta(theta_sd_arr["tool", , ], "Confusion matrix  $\pi^{(3)}$  (posterior SD)")

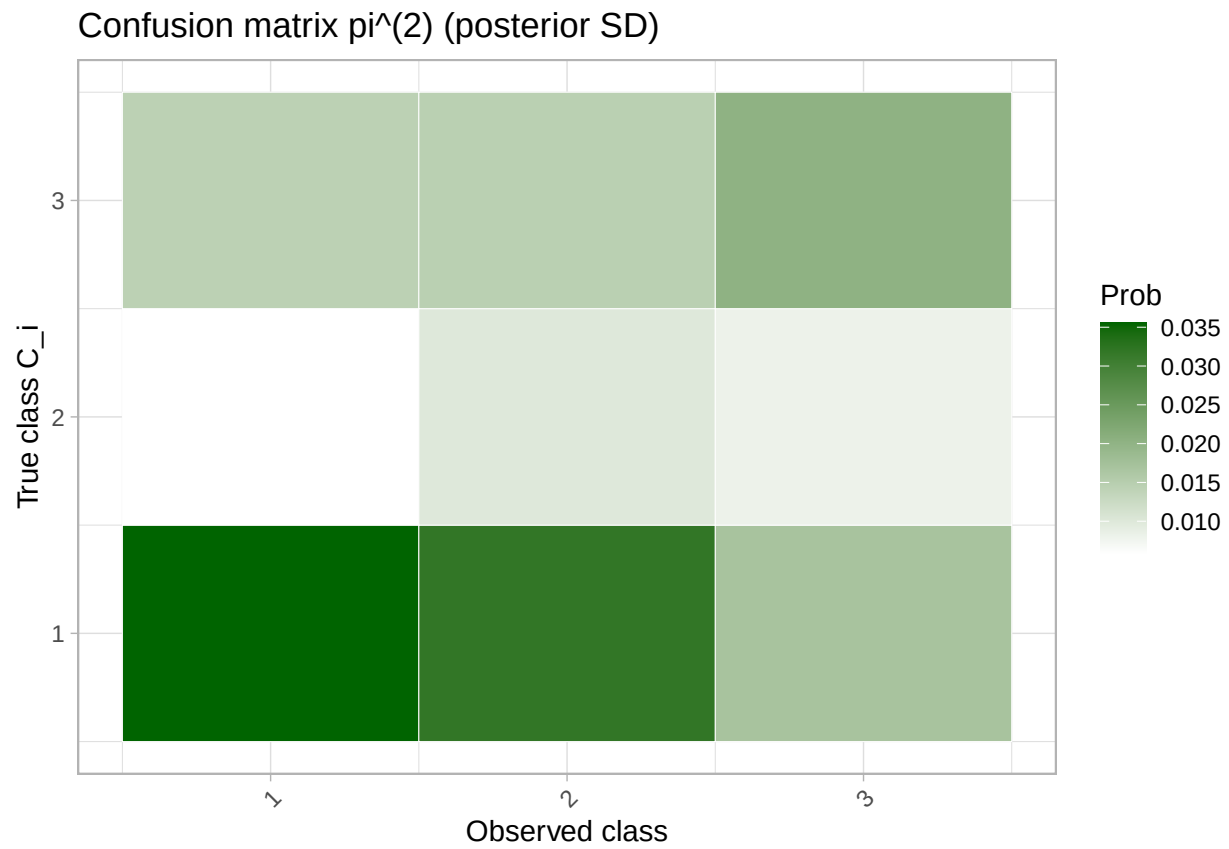
save_plot(p_theta_h1_sd, "theta_human1_sd.png")
save_plot(p_theta_h2_sd, "theta_human2_sd.png")
save_plot(p_theta_tool_sd, "theta_tool_sd.png")

p_theta_h1_sd

```

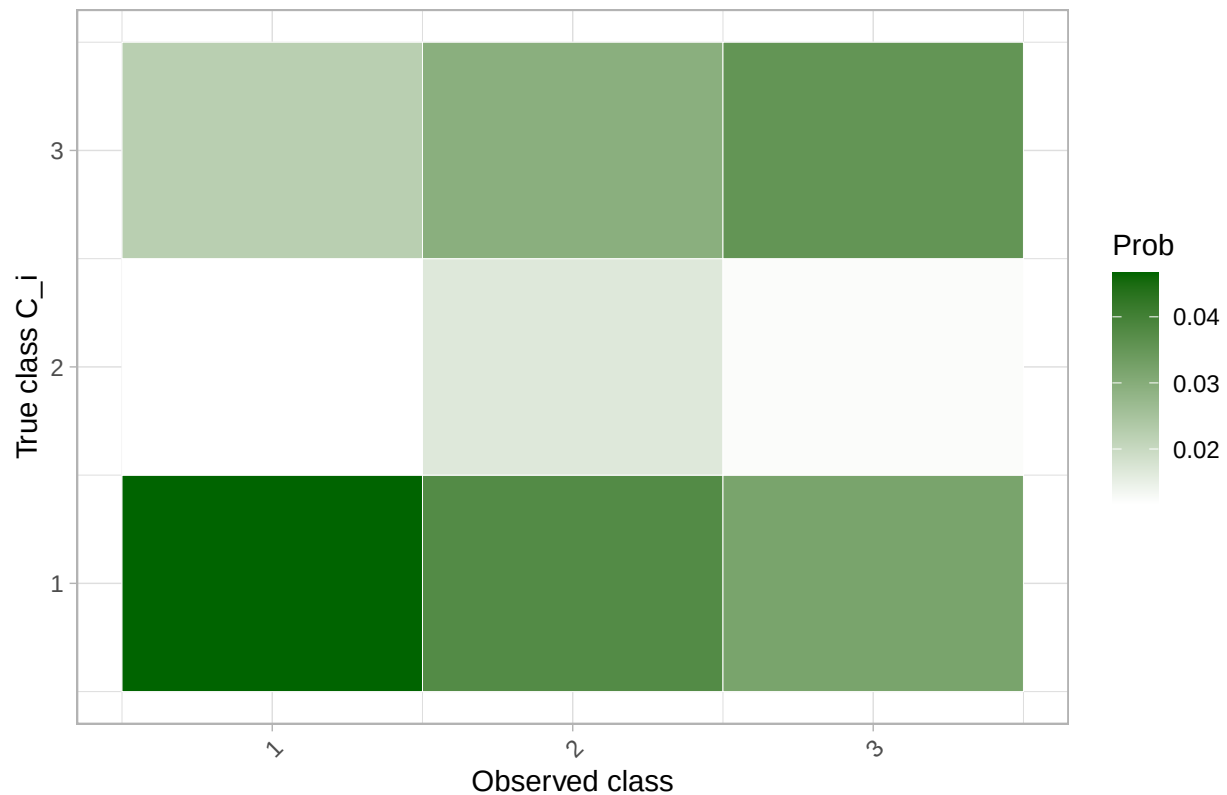


```
p_theta_h2_sd
```



p_theta_tool_sd

Confusion matrix $\pi^{(3)}$ (posterior SD)



```
# -----
# 2) Class prevalence ( $\pi$ ), posterior mean and uncertainty
# -----
pi_summ <- summarise_draws(
  as_draws_df(fit_basis$draws("pi")),
  mean, sd,
  ~quantile(.x, probs = c(`5%` = 0.05, `50%` = 0.50, `95%` = 0.95), na.rm = TRUE)
)
pi_summ
```

```
## # A tibble: 3 x 6
##   variable mean      sd `5%` `50%` `95%`
##   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 pi[1]      0.160 0.0170 0.133 0.160 0.189
## 2 pi[2]      0.583 0.0225 0.547 0.584 0.620
## 3 pi[3]      0.256 0.0198 0.224 0.256 0.289
```

```
pi_means <- colMeans(as_draws_matrix(fit_basis$draws("pi")))
pi_df <- data.frame(
  class = isco_levels,
  prevalence = as.numeric(pi_means)
)
```

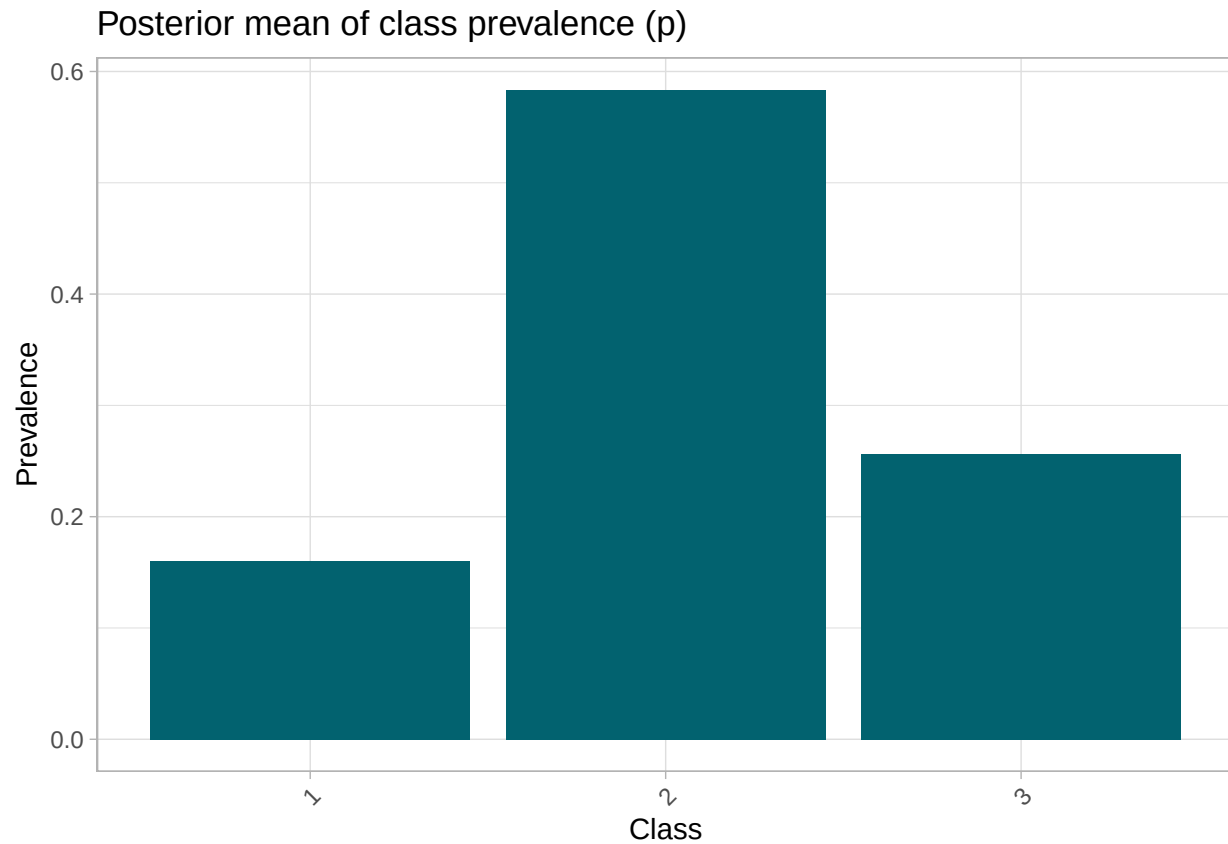
```
p_pi_bar <- ggplot(pi_df, aes(x = class, y = prevalence)) +
  geom_bar(stat = "identity", fill = "#02626f") +
  theme_light() +
  labs(
```

```

title = "Posterior mean of class prevalence (p)",
x = "Class",
y = "Prevalence"
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1))

save_plot(p_pi_bar, "pi_mean_bar.png", width = 6, height = 4)
p_pi_bar

```



```

# -----
# 2b) Rater level scalar summaries from theta
# -----
pi_mean_vec <- as.numeric(pi_means)
names(pi_mean_vec) <- isco_levels

weighted_diag_mass <- function(mat, w) sum(diag(mat) * w)
weighted_offdiag_mass <- function(mat, w) sum((1 - diag(mat)) * w)

rater_summary <- data.frame(
  rater = c("human1", "human2", "tool"),
  weighted_diag_mass = c(
    weighted_diag_mass(theta_human1_mean, pi_mean_vec),
    weighted_diag_mass(theta_human2_mean, pi_mean_vec),
    weighted_diag_mass(theta_tool_mean, pi_mean_vec)
  ),
  weighted_offdiag_mass = c(

```

```

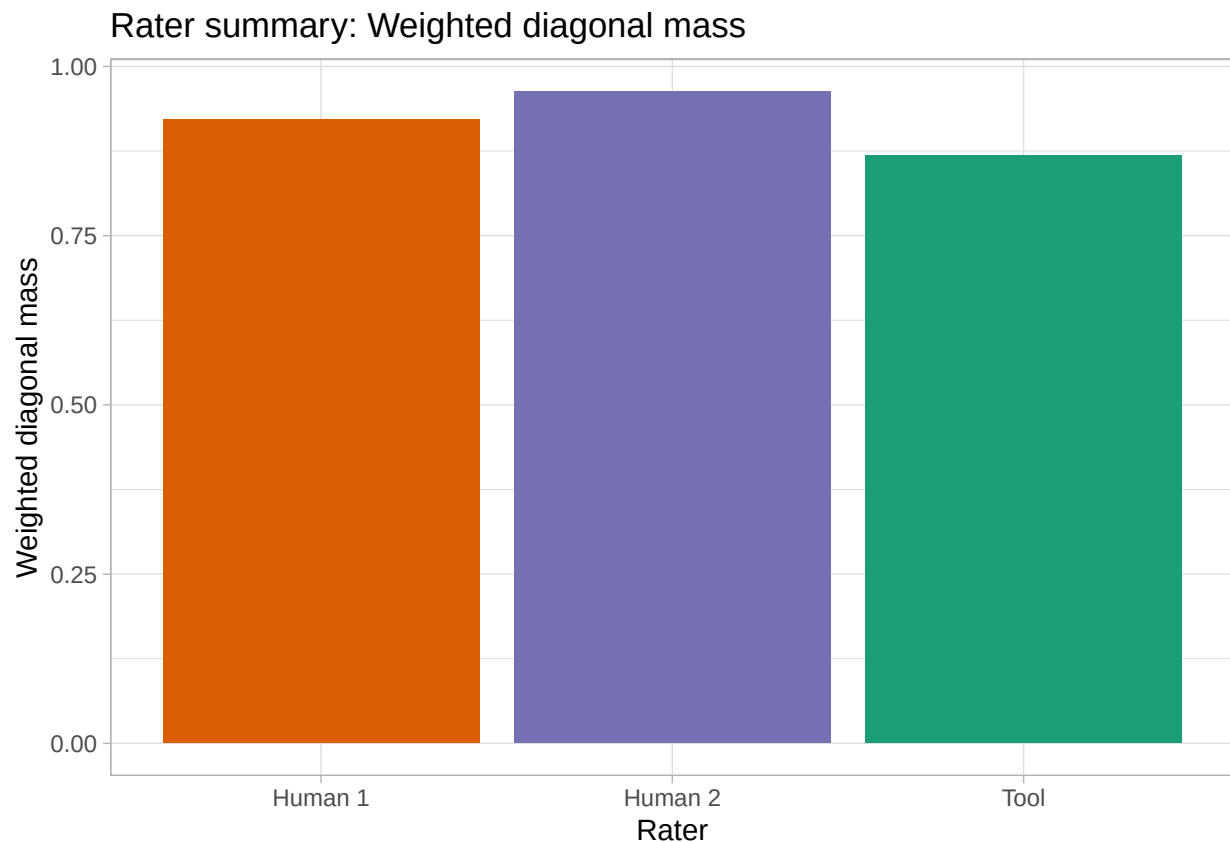
    weighted_offdiag_mass(theta_human1_mean, pi_mean_vec),
    weighted_offdiag_mass(theta_human2_mean, pi_mean_vec),
    weighted_offdiag_mass(theta_tool_mean, pi_mean_vec)
  )
)
rater_summary

##      rater weighted_diag_mass weighted_offdiag_mass
## 1 human1      0.9216081      0.07839185
## 2 human2      0.9630590      0.03694099
## 3  tool      0.8685237      0.13147629

p_rater_diag <- ggplot(rater_summary, aes(x = rater, y = weighted_diag_mass, fill = rater)) +
  geom_bar(stat = "identity") +
  scale_x_discrete(labels = c("human1" = "Human 1", "human2" = "Human 2", "tool" = "Tool")) +
  scale_fill_manual(values = c("human1" = "#d95f02", "human2" = "#7570b3", "tool" = "#1b9e77"),
    guide = "none") +
  theme_light() +
  labs(
    title = "Rater summary: Weighted diagonal mass",
    x = "Rater",
    y = "Weighted diagonal mass"
  )

save_plot(p_rater_diag, "rater_weighted_diag_mass.png", width = 6, height = 4)
p_rater_diag

```



```

# -----
# 3) Posterior class probabilities per item ( $q_{i,j} = P(C_i = j \mid \text{data})$ )
# -----
qz_draws_mat <- fit_basis$draws("q_z", format = "draws_matrix")
qz_mean_vec <- colMeans(qz_draws_mat)

qz_mean <- matrix(
  qz_mean_vec,
  nrow = I, ncol = K,
  dimnames = list(item = item_ids, class = isco_levels)
)

qz_df <- as.data.frame(qz_mean)
p_cols <- paste0("p_", isco_levels)
colnames(qz_df) <- p_cols
qz_df$item_id <- item_ids

# MAP label
qz_df$map_class_idx <- max.col(qz_df[p_cols], ties.method = "first")
qz_df$map_class_label <- isco_levels[qz_df$map_class_idx]

# Uncertainty measures
eps <- 1e-12
qz_df$max_p <- apply(qz_df[p_cols], 1, max)

qz_df$entropy <- -rowSums(as.matrix(qz_df[p_cols]) * log(as.matrix(qz_df[p_cols]) + eps))

qz_df$margin <- apply(qz_df[p_cols], 1, function(p) {
  ps <- sort(p, decreasing = TRUE)
  ps[1] - ps[2]
})

# Credible set per item, smallest set reaching cumulative posterior mass >= level
credible_set <- function(p, classes, level = 0.95) {
  o <- order(p, decreasing = TRUE)
  cum <- cumsum(p[o])
  m <- which(cum >= level)[1]
  classes[o[seq_len(m)]]
}
qz_df$cs95 <- apply(qz_df[p_cols], 1, function(x) credible_set(x, isco_levels, level = 0.95))

table(lengths(qz_df$cs95))

##
## 1 2 3
## 444 9 7

# Plots of uncertainty and confidence
p_maxp_hist <- ggplot(qz_df, aes(x = max_p)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior max probability", y = "Count",
       title = "Model confidence per item") +
  xlim(0, 1.1)

```

```

p_entropy_hist <- ggplot(qz_df, aes(x = entropy)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior entropy", y = "Count",
       title = "Item uncertainty (entropy)") +
  xlim(-0.1, 1.0)

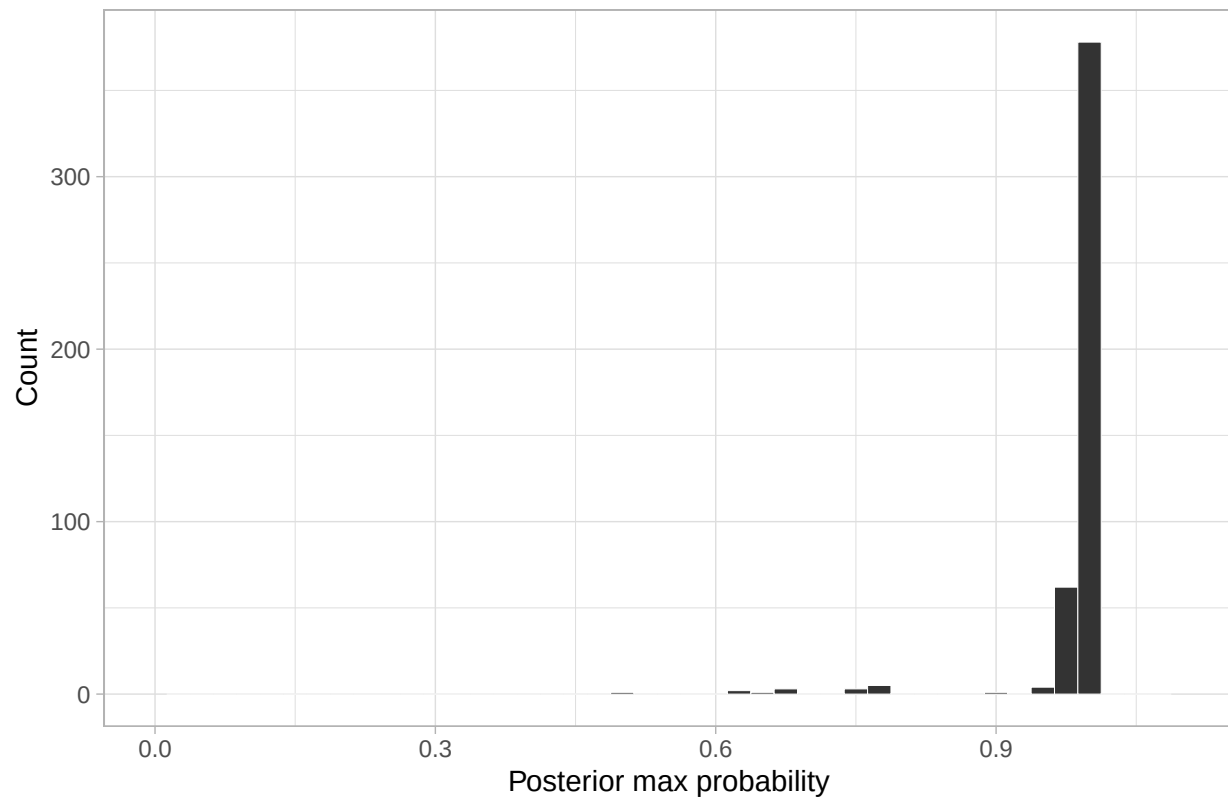
p_margin_hist <- ggplot(qz_df, aes(x = margin)) +
  geom_histogram(binwidth = 0.025, fill = "#333333", color = "white", size = 0.1) +
  theme_light() +
  labs(x = "Posterior margin (top1 minus top2)", y = "Count",
       title = "Item uncertainty (margin)") +
  xlim(0, 1.1)

save_plot(p_maxp_hist, "qz_maxp_hist.png", width = 6, height = 4)
save_plot(p_entropy_hist, "qz_entropy_hist.png", width = 6, height = 4)
save_plot(p_margin_hist, "qz_margin_hist.png", width = 6, height = 4)

p_maxp_hist

```

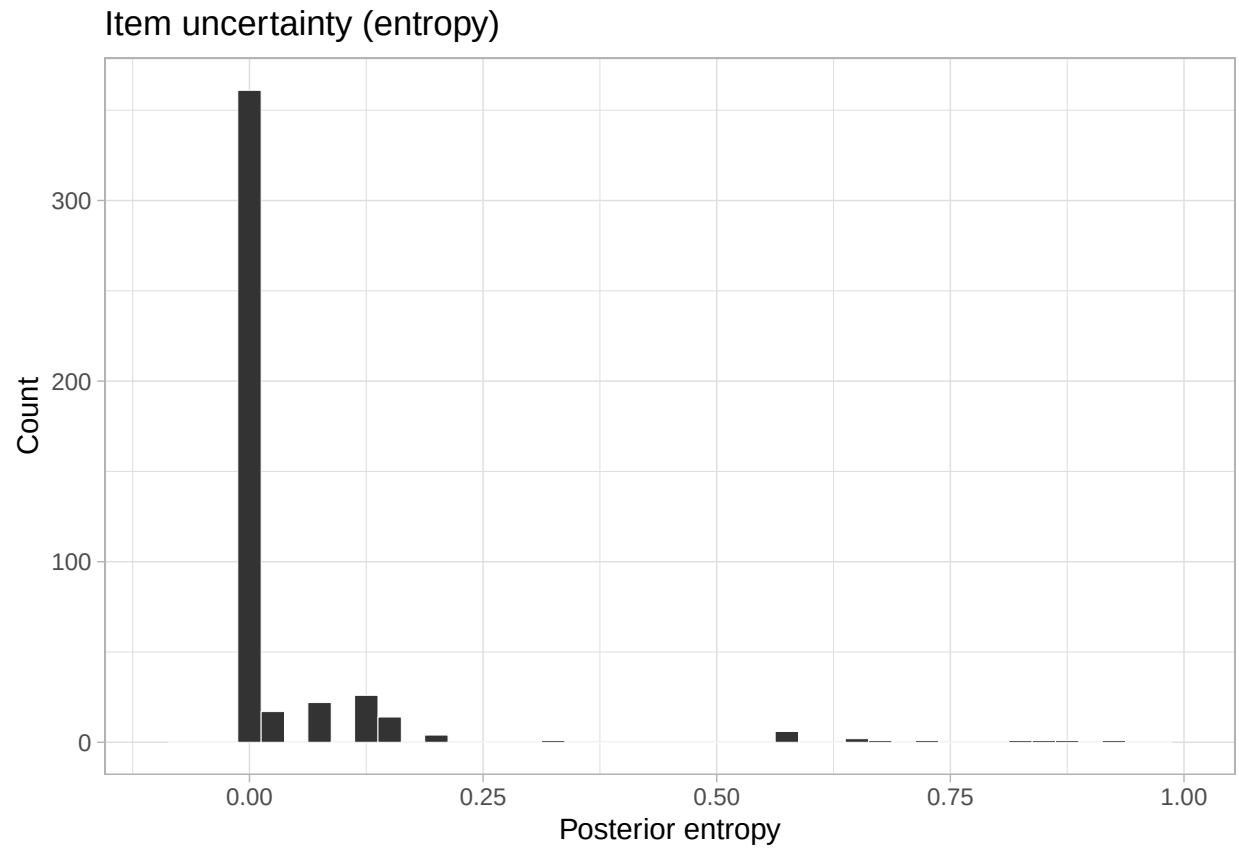
Model confidence per item



```

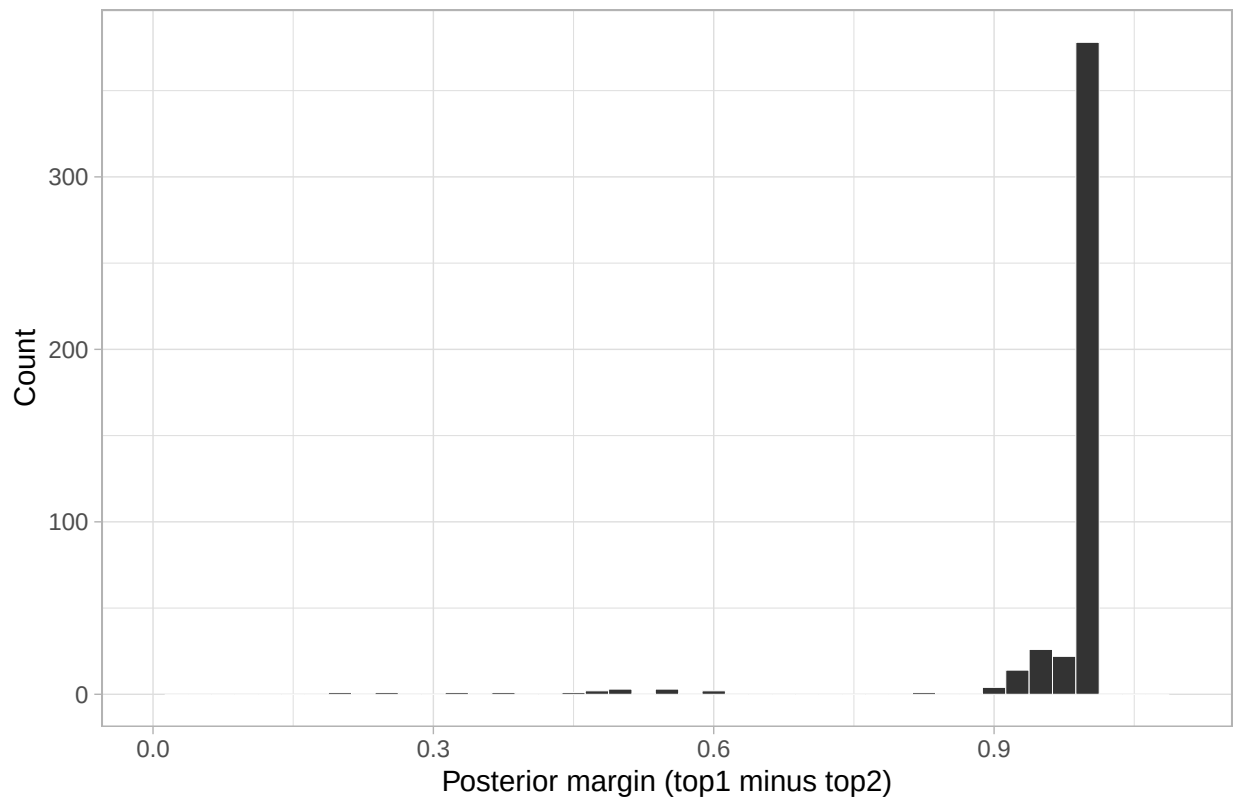
p_entropy_hist

```

p_margin_hist

Item uncertainty (margin)



```
# -----
# 3b) MAP distribution vs pi (sanity check)
# -----
map_tab <- qz_df %>%
  count(map_class_label, name = "n_map") %>%
  mutate(map_share = n_map / sum(n_map)) %>%
  rename(class = map_class_label)

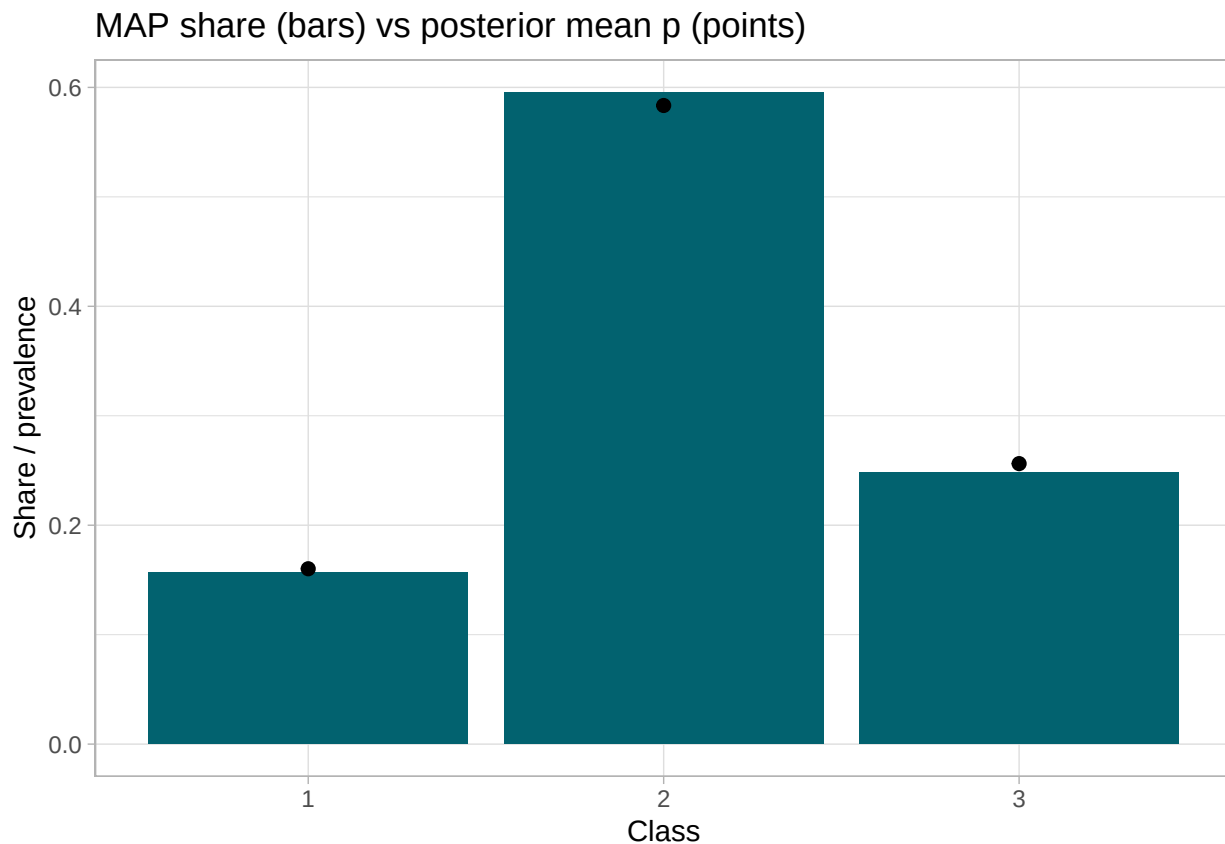
pi_tab <- data.frame(class = isco_levels, pi_mean = as.numeric(pi_means))

map_vs_pi <- left_join(map_tab, pi_tab, by = "class")
map_vs_pi
```

```
##   class n_map map_share  pi_mean
## 1     1     72 0.1565217 0.1602290
## 2     2    274 0.5956522 0.5834762
## 3     3    114 0.2478261 0.2562948
```

```
p_map_vs_pi <- ggplot(map_vs_pi, aes(x = class)) +
  geom_col(aes(y = map_share), fill = "#02626f") +
  geom_point(aes(y = pi_mean), size = 2) +
  theme_light() +
  labs(
    title = "MAP share (bars) vs posterior mean p (points)",
    x = "Class",
    y = "Share / prevalence"
  )
```

```
save_plot(p_map_vs_pi, "map_share_vs_pi.png", width = 6, height = 4)
p_map_vs_pi
```



```
# -----
# 4) Merge predictions with original ratings, include uncertainty
# -----
ratings <- isco_data %>%
  dplyr::select(item_id, isco_human1, isco_human2, isco_tool)

predictions <- qz_df %>%
  dplyr::select(item_id, map_class_label, all_of(p_cols), max_p, entropy, margin, cs95)

ratings_with_predictions <- ratings %>%
  dplyr::left_join(predictions, by = "item_id") %>%
  dplyr::rename(
    predicted_class = map_class_label,
    rating_human1 = isco_human1,
    rating_human2 = isco_human2,
    rating_tool = isco_tool
  ) %>%
  dplyr::arrange(item_id)

head(ratings_with_predictions, 30)
```

```
##      item_id rating_human1 rating_human2 rating_tool predicted_class      p_1      p_2
##      <int>      <char>      <char>      <char>      <char>      <num>      <num>
```

## 1:	1	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 2:	2	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 3:	3	3	1	2	3	3.147660e-01	1.784945e-01	5.0673		
## 4:	4	2	2	3	2	3.690971e-03	9.857851e-01	1.0523		
## 5:	5	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 6:	6	1	1	1	1	9.997528e-01	2.148491e-05	2.2576		
## 7:	7	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 8:	8	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 9:	9	2	2	3	2	3.690971e-03	9.857851e-01	1.0523		
## 10:	10	3	3	3	3	2.719966e-05	8.867162e-05	9.9988		
## 11:	11	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 12:	12	3	3	1	3	2.964764e-03	1.028952e-03	9.9600		
## 13:	13	2	2	1	2	3.557478e-02	9.633696e-01	1.0555		
## 14:	14	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 15:	15	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 16:	16	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 17:	17	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 18:	18	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 19:	19	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 20:	20	3	3	3	3	2.719966e-05	8.867162e-05	9.9988		
## 21:	21	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 22:	22	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 23:	23	1	1	1	1	9.997528e-01	2.148491e-05	2.2576		
## 24:	24	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 25:	25	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 26:	26	2	2	1	2	3.557478e-02	9.633696e-01	1.0555		
## 27:	27	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 28:	28	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 29:	29	2	2	2	2	2.486451e-04	9.996856e-01	6.5709		
## 30:	30	3	3	2	3	3.194105e-04	1.425172e-02	9.8542		
##	item_id	rating_human1	rating_human2	rating_tool	predicted_class	p_1		p_2		
##	max_p	entropy	margin	cs95						
##	<num>	<num>	<num>	<list>						
## 1:	0.9996856	0.003010729	0.9994370	2						
## 2:	0.9996856	0.003010729	0.9994370	2						
## 3:	0.5067395	1.015887762	0.1919735	3,1,2						
## 4:	0.9857851	0.082716657	0.9752612	2						
## 5:	0.9996856	0.003010729	0.9994370	2						
## 6:	0.9997528	0.002373638	0.9995270	1						
## 7:	0.9996856	0.003010729	0.9994370	2						
## 8:	0.9996856	0.003010729	0.9994370	2						
## 9:	0.9857851	0.082716657	0.9752612	2						
## 10:	0.9998841	0.001229153	0.9997955	3						
## 11:	0.9996856	0.003010729	0.9994370	2						
## 12:	0.9960063	0.028321880	0.9930415	3						
## 13:	0.9633696	0.161867437	0.9277948	2						
## 14:	0.9996856	0.003010729	0.9994370	2						
## 15:	0.9996856	0.003010729	0.9994370	2						
## 16:	0.9996856	0.003010729	0.9994370	2						
## 17:	0.9996856	0.003010729	0.9994370	2						
## 18:	0.9996856	0.003010729	0.9994370	2						
## 19:	0.9996856	0.003010729	0.9994370	2						
## 20:	0.9998841	0.001229153	0.9997955	3						
## 21:	0.9996856	0.003010729	0.9994370	2						

```
## 22: 0.9996856 0.003010729 0.9994370      2
## 23: 0.9997528 0.002373638 0.9995270      1
## 24: 0.9996856 0.003010729 0.9994370      2
## 25: 0.9996856 0.003010729 0.9994370      2
## 26: 0.9633696 0.161867437 0.9277948      2
## 27: 0.9996856 0.003010729 0.9994370      2
## 28: 0.9996856 0.003010729 0.9994370      2
## 29: 0.9996856 0.003010729 0.9994370      2
## 30: 0.9854289 0.077617723 0.9711771      3
##      max_p      entropy      margin      cs95
```

```
# Most uncertain items by entropy, include posterior probabilities
```

```
most_uncertain <- ratings_with_predictions %>%
```

```
  dplyr::arrange(desc(entropy)) %>%
```

```
  dplyr::select(item_id, predicted_class, entropy, margin, max_p, all_of(p_cols),
                rating_human1, rating_human2, rating_tool)
```

```
head(most_uncertain, 20)
```

```
##      item_id predicted_class      entropy      margin      max_p      p_1      p_2      p_3 rating
##      <int>      <char>      <num>      <num>      <num>      <num>      <num>      <num>
## 1:         3          3 1.0158878 0.1919735 0.5067395 0.314766005 0.178494514 0.506739480
## 2:        388          1 0.9261127 0.3813776 0.6140968 0.614096829 0.232719199 0.153183973
## 3:        129          1 0.8630559 0.4648996 0.6652136 0.665213642 0.134472350 0.200314008
## 4:         94          3 0.8449870 0.4617865 0.6728206 0.116145316 0.211034079 0.672820605
## 5:        165          2 0.8275399 0.4835983 0.6856060 0.112386182 0.685606035 0.202007783
## 6:        341          2 0.7145177 0.2456009 0.6173977 0.371796824 0.617397684 0.010805492
## 7:        365          3 0.6749803 0.3171319 0.6554740 0.006183813 0.338342140 0.655474047
## 8:        327          3 0.6621982 0.5919586 0.7699906 0.178032052 0.051977318 0.769990630
## 9:        368          3 0.6621982 0.5919586 0.7699906 0.178032052 0.051977318 0.769990630
## 10:       177          2 0.5859610 0.5033809 0.7493894 0.004602115 0.749389404 0.246008482
## 11:       301          2 0.5859610 0.5033809 0.7493894 0.004602115 0.749389404 0.246008482
## 12:       394          2 0.5859610 0.5033809 0.7493894 0.004602115 0.749389404 0.246008482
## 13:       183          2 0.5672232 0.5385349 0.7667775 0.228242572 0.766777454 0.004979974
## 14:       260          2 0.5672232 0.5385349 0.7667775 0.228242572 0.766777454 0.004979974
## 15:       452          2 0.5672232 0.5385349 0.7667775 0.228242572 0.766777454 0.004979974
## 16:       421          3 0.3248033 0.8165502 0.9065966 0.090046368 0.003357057 0.906596575
## 17:       195          1 0.1991602 0.9097255 0.9534176 0.953417585 0.043692061 0.002890354
## 18:       214          1 0.1991602 0.9097255 0.9534176 0.953417585 0.043692061 0.002890354
## 19:       359          1 0.1991602 0.9097255 0.9534176 0.953417585 0.043692061 0.002890354
## 20:       415          1 0.1991602 0.9097255 0.9534176 0.953417585 0.043692061 0.002890354
##      item_id predicted_class      entropy      margin      max_p      p_1      p_2      p_3 rating
##      rating_human2 rating_tool
##      <char>      <char>
## 1:         1          2
## 2:         2          3
## 3:         1          3
## 4:         3          1
## 5:         2          1
## 6:         1          2
## 7:         2          3
## 8:         3          2
## 9:         3          2
## 10:        3          2
## 11:        3          2
```

```

## 12:          3          2
## 13:          2          2
## 14:          2          2
## 15:          2          2
## 16:          1          3
## 17:          2          1
## 18:          2          1
## 19:          2          1
## 20:          2          1
##      rating_human2 rating_tool

# -----
# 5) Observed agreement, descriptive, not model based
# -----
mean(isco_data$isco_human1 == isco_data$isco_tool, na.rm = TRUE)

## [1] 0.8065217

mean(isco_data$isco_human2 == isco_data$isco_tool, na.rm = TRUE)

## [1] 0.8456522

isco_data %>%
  dplyr::count(isco_human1, isco_human2, isco_tool) %>%
  dplyr::arrange(desc(n))

##      isco_human1 isco_human2 isco_tool      n
##      <char>      <char>      <char> <int>
## 1:          2          2          2    235
## 2:          3          3          3     80
## 3:          1          1          1     46
## 4:          2          2          3     11
## 5:          3          2          2     11
## 6:          3          3          2     11
## 7:          1          1          2     10
## 8:          2          2          1      9
## 9:          3          3          1      7
## 10:         1          1          3      6
## 11:         1          3          3      5
## 12:         2          3          3      5
## 13:         1          2          1      4
## 14:         2          1          1      4
## 15:         1          2          2      3
## 16:         2          3          2      3
## 17:         1          3          2      2
## 18:         1          2          3      1
## 19:         2          1          2      1
## 20:         2          1          3      1
## 21:         2          3          1      1
## 22:         3          1          2      1
## 23:         3          1          3      1
## 24:         3          2          1      1
## 25:         3          2          3      1
##      isco_human1 isco_human2 isco_tool      n

# -----
# Diagnose: Konvergenz, Divergences, Label switching (pi, theta)

```

```

# -----

library(posterior)

# 0) Grunddiagnostik aus cmdstanr
cat("\n--- cmdstanr diagnostic_summary() ---\n")

##
## --- cmdstanr diagnostic_summary() ---
print(tryCatch(fit_basis$diagnostic_summary(), error = function(e) e))

## $num_divergent
## [1] 0 0 0 0
##
## $num_max_tredepth
## [1] 0 0 0 0
##
## $ebfmi
## [1] 0.9086901 0.9559382 0.9398511 0.9236251
cat("\n--- cmdstan_diagnose() (falls verfügbar) ---\n")

##
## --- cmdstan_diagnose() (falls verfügbar) ---
print(tryCatch(fit_basis$cmdstan_diagnose(), error = function(e) e))

## Checking sampler transitions treedepth.
## Treedepth satisfactory for all transitions.
##
## Checking sampler transitions for divergences.
## No divergent transitions found.
##
## Checking E-BFMI - sampler transitions HMC potential energy.
## E-BFMI satisfactory.
##
## Rank-normalized split effective sample size satisfactory for all parameters.
##
## Rank-normalized split R-hat values satisfactory for all parameters.
##
## Processing complete, no problems detected.
## $status
## [1] 0
##
## $stdout
## [1] "Checking sampler transitions treedepth.\r\nTreedepth satisfactory for all transitions.\r\n\r\nC
##
## $stderr
## [1] ""
##
## $timeout
## [1] FALSE

# 1) R-hat Check nur für pi und theta (hart: > 1.1)
s <- fit_basis$summary()

```

```

s$rhat <- as.numeric(s$rhat)

bad_rhat <- subset(s, rhat > 1.1 & grepl("(pi|theta)\\b", variable))
cat("\n--- Parameters with R-hat > 1.1 (pi/theta) ---\n")

##
## --- Parameters with R-hat > 1.1 (pi/theta) ---
if (nrow(bad_rhat) == 0) {
  cat("None\n")
} else {
  print(bad_rhat[order(-bad_rhat$rhat), c("variable", "rhat", "ess_bulk", "ess_tail")], row.names = FALSE)
}

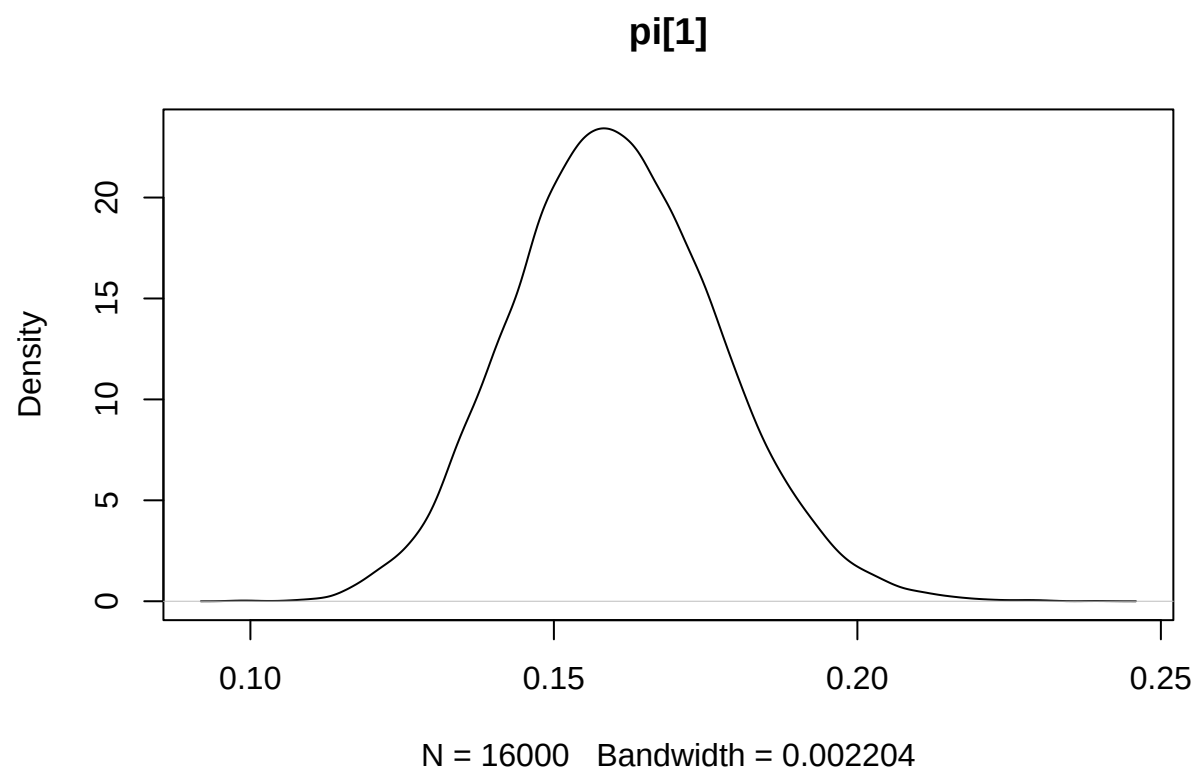
## None
# 2) pi Draws: Spaltennamen prüfen + Dichten plotten
pi_df <- as_draws_df(fit_basis$draws("pi"))
cat("\n--- pi columns ---\n")

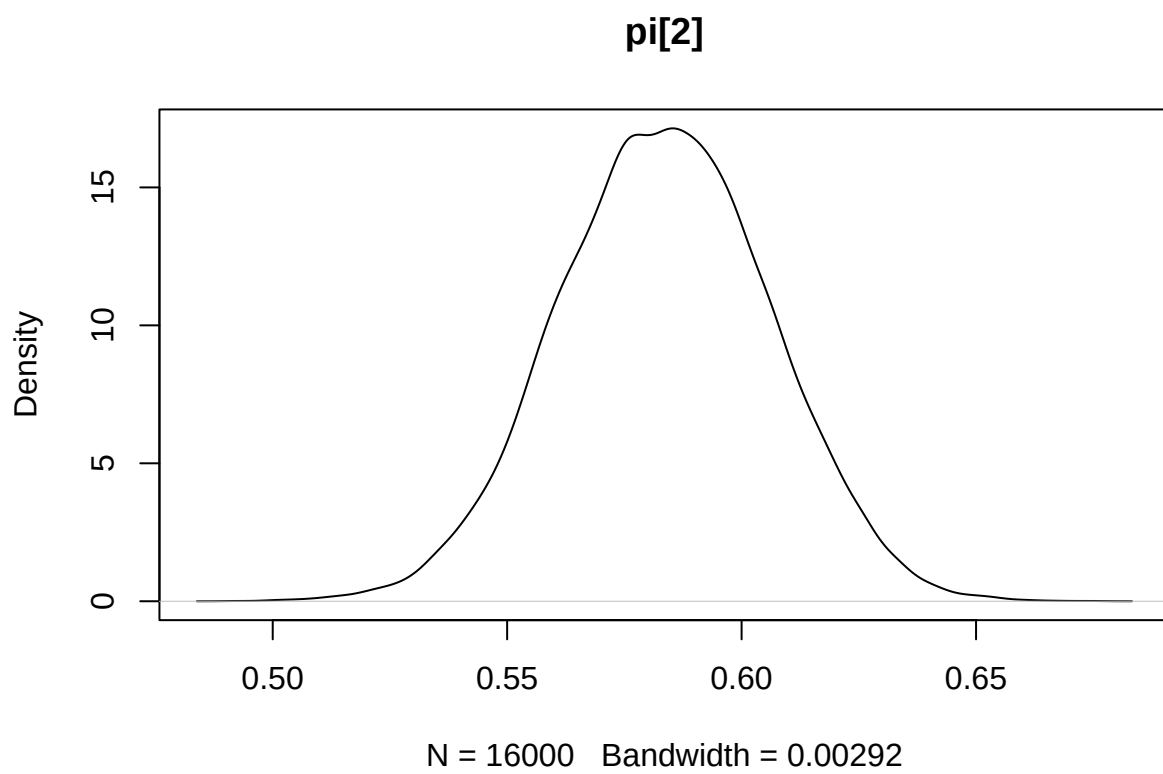
##
## --- pi columns ---
print(names(pi_df))

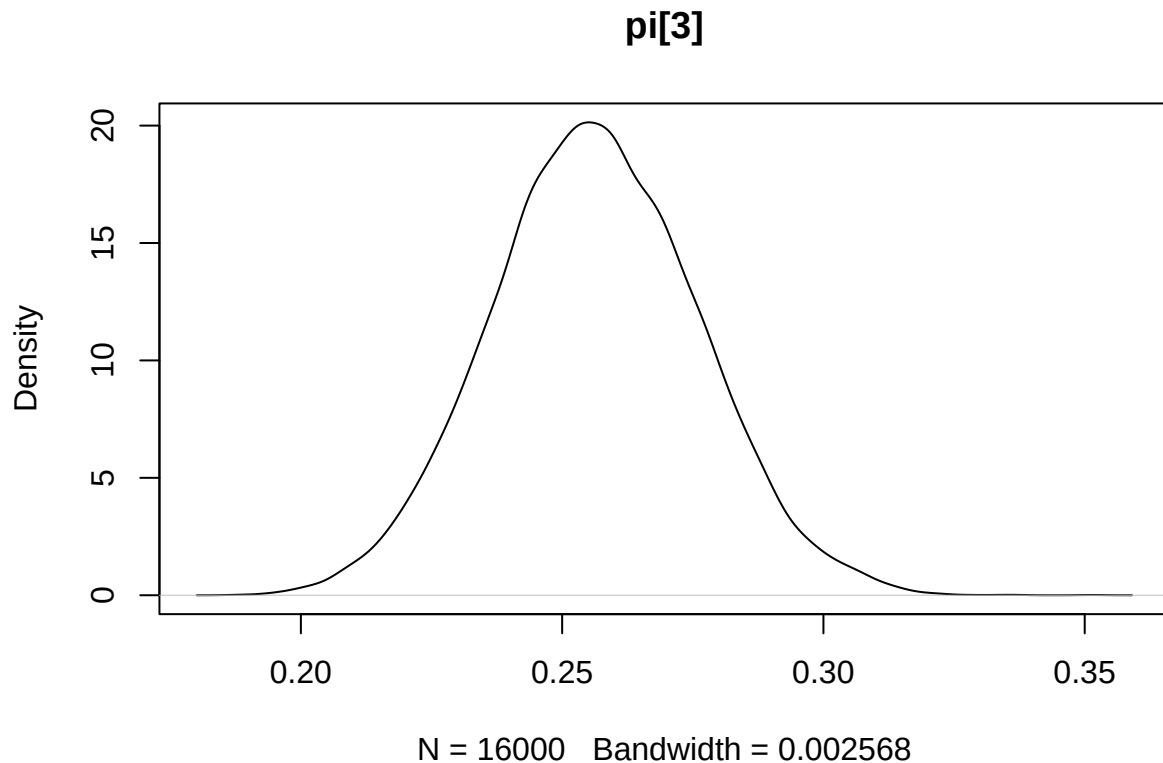
## [1] "pi[1]"      "pi[2]"      "pi[3]"      ".chain"      ".iteration" ".draw"
# Dichteplots für pi[1..K]
K <- length(grep("^pi\\[", names(pi_df)))
if (K == 0) stop("No pi columns found in pi_df, something is off.")

for (k in 1:K) {
  col <- paste0("pi[", k, "]")
  if (!col %in% names(pi_df)) next
  plot(density(pi_df[[col]]), main = col)
}

```





```
# 3) Label switching Check: Chain-Mittelwerte für pi
# Wenn Chains permutiert sind (zB Chain1: 0.7/0.2/0.1, Chain2: 0.2/0.7/0.1),
# dann ist das Label switching.
pi_arr <- fit_basis$draws("pi", format = "draws_array") # iter x chain x param
pi_chain_means <- apply(pi_arr, c(2, 3), mean)
colnames(pi_chain_means) <- paste0("pi[", 1:ncol(pi_chain_means), "]")

cat("\n--- pi chain means (chain x component) ---\n")
```

```
##
## --- pi chain means (chain x component) ---
print(round(pi_chain_means, 4))
```

```
##      variable
## chain pi[1] pi[2] pi[3]
##    1 0.1603 0.5838 0.2559
##    2 0.1603 0.5835 0.2562
##    3 0.1599 0.5834 0.2567
##    4 0.1604 0.5833 0.2563
```

```
# 4) Optional: gleiche Idee für diagonale von theta (guter Indikator)
# theta hat Spalten wie theta[<rater>,<>true>,<given>]
theta_df <- as_draws_df(fit_basis$draws("theta"))
theta_cols <- names(theta_df)

# Diagonaleinträge sammeln: theta[j,k,k]
```

```

diag_cols <- character(0)
for (j in 1:J) {
  for (k in 1:K) {
    diag_cols <- c(diag_cols, paste0("theta[", j, ",", k, ",", k, ", ]"))
  }
}
diag_cols <- diag_cols[diag_cols %in% theta_cols]

cat("\n--- Found theta diagonal columns ---\n")

##
## --- Found theta diagonal columns ---
print(diag_cols)

## [1] "theta[1,1,1]" "theta[1,2,2]" "theta[1,3,3]" "theta[2,1,1]" "theta[2,2,2]" "theta[2,3,3]" "theta[3,1,1]"
## [8] "theta[3,2,2]" "theta[3,3,3]"

if (length(diag_cols) > 0) {
  theta_diag_arr <- fit_basis$draws(diag_cols, format = "draws_array") # iter x chain x param
  theta_diag_chain_means <- apply(theta_diag_arr, c(2, 3), mean)
  colnames(theta_diag_chain_means) <- diag_cols

  cat("\n--- theta diagonal chain means (chain x diag entry) ---\n")
  print(round(theta_diag_chain_means, 4))
}

##
## --- theta diagonal chain means (chain x diag entry) ---
##      variable
## chain theta[1,1,1] theta[1,2,2] theta[1,3,3] theta[2,1,1] theta[2,2,2] theta[2,3,3] theta[3,1,1] theta[3,2,2] theta[3,3,3]
##      1      0.9117      0.9417      0.8824      0.9088      0.9794      0.9600      0.7665      0.9117      0.9417
##      2      0.9110      0.9411      0.8828      0.9087      0.9793      0.9603      0.7660      0.9110      0.9411
##      3      0.9112      0.9418      0.8820      0.9081      0.9793      0.9601      0.7664      0.9112      0.9418
##      4      0.9118      0.9417      0.8828      0.9081      0.9796      0.9596      0.7670      0.9118      0.9417
##      variable
## chain theta[3,3,3]
##      1      0.8176
##      2      0.8176
##      3      0.8177
##      4      0.8180

# 5) Quick verdict helper (nur Heuristik)
# Wenn pi_chain_means je Chain eine Permutation der gleichen Werte ist, dann Label switching sehr wahrs
cat("\n--- Heuristic verdict ---\n")

##
## --- Heuristic verdict ---
cat("If pi chain means look like permutations across chains, you have label switching.\n")

## If pi chain means look like permutations across chains, you have label switching.
cat("If divergences/treedepth hits appear, you have sampler geometry issues (then adapt_delta/max_treedepth)\n")

## If divergences/treedepth hits appear, you have sampler geometry issues (then adapt_delta/max_treedepth)

```